arXiv:2601.18811v2 [cs.LG] 28 Jan 2026

Variational Quantum Circuit-Based Reinforcement Learning for Dynamic Portfolio Optimization

A Preprint

Vincent Gurgul
Chair of Information Systems
Humboldt University Berlin
Unter d. Linden 6, 10117 Berlin

Ying Chen
Department of Mathematics
National University of Singapore
2 Science Dr., Singapore 117543

Stefan Lessmann
Chair of Information Systems
Humboldt University Berlin
Unter d. Linden 6, 10117 Berlin
Bucharest University of Economic Studies
6 Piata Romana, 010374, Romania

January 29, 2026

Abstract

This paper presents a Quantum Reinforcement Learning (QRL) solution to the dynamic portfolio optimization problem based on Variational Quantum Circuits. The implemented QRL approaches are quantum analogues of the classical neural-network-based Deep Deterministic Policy Gradient and Deep Q-Network algorithms. Through an empirical evaluation on real-world financial data, we show that our quantum agents achieve risk-adjusted performance comparable to, and in some cases exceeding, that of classical Deep RL models with several orders of magnitude more parameters. However, while quantum circuit execution is inherently fast at the hardware level, practical deployment on cloud-based quantum systems introduces substantial latency, making end-to-end runtime currently dominated by infrastructural overhead and limiting practical applicability. Taken together, our results suggest that QRL is theoretically competitive with state-of-the-art classical reinforcement learning and may become practically advantageous as deployment overheads diminish. This positions QRL as a promising paradigm for dynamic decision-making in complex, high-dimensional, and non-stationary environments such as financial markets. The complete codebase is released as open source at: `https://github.com/VincentGurgul/qrl-dpo-public`

Keywords Quantum Machine Learning · Quantum Reinforcement Learning · Near-Term Quantum Algorithms · Portfolio Optimization · Time Series Analysis

1 Introduction

Portfolio optimization is a cornerstone of finance, aiming to allocate assets in a way that maximizes returns while minimizing risk. Traditional methods like Mean-Variance Optimization (MVO) have long been the standard approach for static portfolio allocation. However, since such static methods treat portfolio optimization as a one-shot problem (an allocation is performed at single point in time based on past data), they often fail to capture the dynamic and sequential nature of real-world investment scenarios where market conditions are constantly evolving.

Recent advances in quantum computing offer potential for solving complex optimization problems more efficiently than classical counterparts. Quantum Annealing and Variational Quantum Algorithms have shown

promise in addressing static portfolio optimization formulations through the Quadratic Unconstrained Binary Optimization (QUBO) framework. While these quantum methods have demonstrated improvements over classical algorithms in solving QUBOs, the QUBO is also a static problem formulation and does not naturally enable adaptive decision-making over time.

Reinforcement Learning (RL), a branch of machine learning that deals with sequential decision-making, has shown significant potential in dynamic portfolio management by facilitating the learning of optimal policies through interaction with the environment. However, while powerful, RL faces challenges in terms of scalability and training efficiency in the case of large state-action spaces. Quantum Reinforcement Learning (QRL) merges RL techniques with quantum computing, offering the potential for faster convergence and more efficient policy representation compared to classical methods.

Despite the growing interest in quantum technologies, there remains a notable gap in applying them to dynamic portfolio optimization. Most existing studies in the quantum realm either treat portfolio problems as a static QUBO and solve it using quantum solvers, or explore QRL on simple control tasks without addressing complex real-world applications like sequential investment decisions.

This paper aims to bridge this gap by introducing a novel QRL framework for dynamic portfolio optimization and systematically benchmarking it against classical baselines. We propose a fully quantum RL approach where the agent learns an optimal trading policy based on a hybrid optimization loop over a dynamic state-action space, while harnessing the entanglement-induced expressivity and potential for parallelism that is inherent to Variational Quantum Circuits (VQCs).

By integrating quantum computing into a sequential financial decision-making process, we demonstrate how this technology can enhance and accelerate traditional portfolio management practices and pave the way for more advanced investment strategies, but also highlight the current limitations of quantum hardware and cloud computing solutions.

2 Literature Review

In the following we contextualize our work within the landscape of portfolio optimization, RL, and quantum machine learning by examining the evolution and current state of research across these intersecting fields. We summarize foundational methods in classical portfolio theory, highlight recent advances in quantum algorithms for static optimization, and identify the emerging role of QRL. By identifying gaps in the exisiting literature we motivate our proposed QRL framework for portfolio optimization.

Portfolio optimization is a fundamental problem in finance, describing the process of allocating capital across a set of financial assets. This objective is typically formulated as identifying the optimal portfolio weights $\mathbf{w} = [w_1, \ldots, w_N]$ over N assets to balance expected return and risk. The majority of modern portfolio optimization methodologies build on the foundational model of Markowitz [1952], in which the balance between expected return and portfolio risk is expressed as a constrained quadratic optimization problem:

$$\max_{\mathbf{w}} \quad \boldsymbol{\mu}^\top \mathbf{w} - \eta\, \mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w} \quad \text{s.t.} \quad \sum_{i=1}^{N} w_i = 1,\ w_i \geq 0,\ \eta \in [0,1]\ , \tag{2.1}$$

$$\text{where} \quad \boldsymbol{\mu}^\top \mathbf{w} : \text{ portfolio return,}$$

$$\mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w} : \text{ portfolio variance (risk).}$$

In the Black–Litterman model, the Markowitz formulation is extended by replacing the return vector $\boldsymbol{\mu}$ with a posterior estimate $\boldsymbol{\mu}_{\mathrm{BL}}$, which combines equilibrium market returns with subjective investor views [Black and Litterman, 1992]. This approach stabilizes portfolio optimization by reducing the sensitivity to input return estimates while enabling the explicit incorporation of informed opinions. The posterior mean of expected returns is given by

$$\boldsymbol{\mu}_{\mathrm{BL}} = \boldsymbol{\pi} + \tau \boldsymbol{\Sigma} P^{\top} (P \tau \boldsymbol{\Sigma} P^{\top} + \Omega)^{-1} (\mathbf{q} - P \boldsymbol{\pi}), \tag{2.2}$$

where $\boldsymbol{\pi} = \lambda \boldsymbol{\Sigma} \mathbf{w}_m$: implied equilibrium returns,

P : view matrix defining assets involved in each investor view,

$\mathbf{q}$: vector of expected returns associated with investor views,

Ω : covariance matrix representing uncertainty of the views,

τ : scalar reflecting the uncertainty of the prior equilibrium returns.

For experienced investors, this framework allows the integration of domain knowledge and market intuition into quantitative allocation. However, in empirical or data-driven applications, a key challenge lies in quantitatively inferring coherent "views" from market sentiment or alternative data sources, introducing an additional layer of model complexity.

The Markowitz Constrained Quadratic Model (CQM) can also easily be extended to include additional constraints, such as limits on individual asset weights, sector exposures, transaction costs, CO_2 emissions, or ESG criteria [Fabozzi et al., 2012]. Salirrosas et al. [2025] extend the classic Markowitz formulation with binary asset weights by adding a market sentiment variable that increases diversification during "anxious" market sentiment periods, but find no advantage over the standard formulation.

Classical static optimization methods have a long history of being used to solve these problems [Salo et al., 2024]. However, dynamically rebalancing a portfolio over time (e.g. across many trading periods) is a complex decision-making task that challenges classical optimization techniques. Recent advances in RL offer powerful data-driven solutions for such dynamic portfolio management. Deep neural network-based RL algorithms, particularly Deep Q-Networks (DQNs), have achieved state-of-the-art performance in portfolio trading and asset allocation, outperforming traditional strategies in terms of both Sharpe ratio and cumulative returns [Zhang et al., 2020].

At the same time, quantum computing has emerged as a promising paradigm for tackling computationally hard optimization problems in finance. Leveraging superposition and entanglement, quantum algorithms can explore vast solution spaces and identify high-quality approximate solutions to NP-hard optimization problems, within practical runtime limits. While many of these approaches were initially described only theoretically, recent advances in Quantum Processing Units (QPUs) have enabled the implementation on real hardware, albeit with limitations regarding noise and qubit counts [Orús et al., 2019, Egger et al., 2020, Buonaiuto et al., 2023].

QRL lies at the intersection of these trends, aiming to integrate quantum computing into the RL framework. It is an emerging field with the potential to enhance learning and decision-making processes by using quantum circuits as function approximators or by accelerating parts of RL algorithms [Meyer et al., 2024]. In this section, we review prior work on quantum computing for portfolio optimization and on QRL, followed by a synthesis of the key research gaps that motivate our study.

2.1 Quantum Portfolio Optimization

Recent reviews highlight three main strands of research in quantum portfolio optimization: the Variational Quantum Eigensolver (VQE), the Quantum Approximate Optimization Algorithm (QAOA), and Quantum Annealing (QA) approaches, particularly those implemented on D-Wave hardware [Yulianti and Surendro, 2022, Sehrawat, 2024, Volpe et al., 2025].

A unified mathematical formulation underlies all three of these approaches: they require the optimization to be expressed as a QUBO problem [Buonaiuto et al., 2023]. A QUBO is a quadratic objective function defined over binary decision variables without explicit constraints:

$$\min_{\mathbf{b}} \quad \mathbf{b}^\top \mathbf{Q}\, \mathbf{b} + \mathbf{c}^\top \mathbf{b} \ ,$$

$$\begin{aligned} \text{where} \quad & \mathbf{b} \in \{0,1\}^N : \text{ binary decision variables,} \\ & \mathbf{Q} \in \mathbb{R}^{N \times N} : \text{ symmetric matrix of quadratic coefficients,} \\ & \mathbf{c} \in \mathbb{R}^N : \text{ vector of linear coefficients.} \end{aligned}$$

Despite its apparent simplicity, the QUBO formalism is remarkably general. Any problem defined over a finite set of discrete variables, including all NP-hard combinatorial optimization problems, can be reformulated exactly in this structure. Continuous or mixed-integer problems can be approximated through binary encodings or quadratic penalty functions that emulate continuous behaviour [Kochenberger et al., 2014, Glover et al., 2019].

Portfolio optimization can be formulated in QUBO form through two approaches, both static in nature. The first treats portfolio optimization as a combinatorial selection problem, where the goal is to select a subset of assets from a larger pool to maximize return and minimize risk. This involves defining binary decision variables b_i that indicate whether asset i is included in the portfolio ($b_i = 1$) or not ($b_i = 0$). Alternatively, it can be viewed as going long or short on each asset, with $b_i = 1$ indicating a long position and $b_i = 0$ indicating a short position. The objective function can then be written as a constrained quadratic function of these binary variables, as expressed in Eq. 2.3, incorporating expected returns and covariances between asset returns [Phillipson and Bhatia, 2021, Brandhofer et al., 2022].

$$\max_{\mathbf{b}} \quad \boldsymbol{\mu}^\top \mathbf{b} - \eta\, \mathbf{b}^\top \boldsymbol{\Sigma}\, \mathbf{b} \quad \text{s.t.} \quad \sum_{i=1}^{N} b_i = K \ , \tag{2.3}$$

$$\begin{aligned} \text{where} \quad & \mathbf{b} \in \{0,1\}^N : \text{ binary asset selection vector,} \\ & b_i = 1 \text{ if asset } i \text{ is included in the portfolio, else 0,} \\ & \eta \in \mathbb{R} : \text{ risk aversion parameter,} \\ & \boldsymbol{\mu}^\top \mathbf{b} : \text{ expected return of selected assets,} \\ & \mathbf{b}^\top \boldsymbol{\Sigma}\, \mathbf{b} : \text{ variance (risk) of the selected portfolio,} \\ & K : \text{ predefined number of assets to be selected.} \end{aligned}$$

Since quantum algorithms such as the VQE, QAOA, and QA operate on unconstrained objective functions, the cardinality constraint must be incorporated as a quadratic penalty term, yielding the unconstrained (QUBO) formulation in Eq. 2.4.

$$\max_{\mathbf{b}} \ \boldsymbol{\mu}^\top \mathbf{b} - \eta\, \mathbf{b}^\top \boldsymbol{\Sigma}\, \mathbf{b} - \lambda \left(\sum_{i=1}^{N} b_i - K \right)^2 \ , \tag{2.4}$$

where $\lambda \in \mathbb{R}_{>0}$ is a penalty coefficient ensuring approximate satisfaction of the constraint.

The second approach treats portfolio optimization as a multi-objective problem of allocating fractions of a portfolio value – expressing the portfolio problem as a CQM (the classic Markowitz formulation, see Eq. 2.1).

This formulation can be discretized by replacing continuous asset weights w_i with the closest integer asset amount n_i corresponding to the price of the asset divided by the amount of capital to be invested. Hence, the following binarization approach applies also to discrete Markowitz formulations.

With the help of a binary conversion matrix $\mathbf{C}$, the discrete/discretized integer portfolio optimization can be encoded into binary decision variables, allowing the problem to be represented as a binary quadratic optimization problem. Finally, to arrive at QUBO form, the constraints need to be included directly in the optimization as a penalty term [Buonaiuto et al., 2023]. The resulting formulation is expressed in Eq. 2.5.

$$\max_{\mathbf{b}} \quad (\mathbf{p}^* \circ \boldsymbol{\mu})^\top \mathbf{C}\,\mathbf{b} - \eta\,\mathbf{b}^\top \mathbf{C}^\top \left((\mathbf{p}^* \circ \boldsymbol{\Sigma})^\top \circ \mathbf{p}^*\right) \mathbf{C}\,\mathbf{b} - \lambda((\mathbf{C}^\top \mathbf{p}^*)^\top \mathbf{b} - 1)\ , \tag{2.5}$$

$$\text{where} \quad \mathbf{b} \in \{0,1\}^M\ ,\ M = \sum_{i=1}^{N} d_i\ :\ \text{binarized target vector}\,,$$

$$\lambda \in \mathbb{R}_{>0}\ :\ \text{penalty coefficient for budget constraint}\,,$$

$$d_i = Int(\log_2(n_i^{\max}))\ \ \text{where}\ \ \mathbf{n}:\ \text{integer asset amounts}\,,$$

$$\mathbf{C} = \begin{pmatrix} 2^0 & \cdots & 2^{d_1} & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 2^0 & \cdots & 2^{d_2} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 2^{d_N} \end{pmatrix},$$

$$\mathbf{p}^* = \frac{\mathbf{p}}{B}\ :\ \text{relative asset prices}$$

$$\mathbf{p} \in \mathbb{R}^N\ :\ \text{asset prices}\,,\quad B \in \mathbb{R}\ :\ \text{portfolio budget}\,.$$

Once expressed in any of the mentioned QUBO forms, the objective can be converted into a set of quantum operators (Ising Hamiltonian) and subsequently solved using different quantum paradigms [Egger et al., 2020], such as QAOA, VQE, and QA. An overview of existing literature on quantum portfolio optimization, including problem formulations, approaches, and implementation details, is provided in Table 2.1.

The QAOA alternates between two parameterized quantum operators: a cost Hamiltonian encoding the objective function and a mixer Hamiltonian that explores the feasible space [Farhi et al., 2014]. By iteratively adjusting the parameters of these unitaries through a classical optimization loop, QAOA can approximate a low-energy configuration that corresponds to the optimal portfolio allocation, although the restrictive assumptions under which such behaviour could be formally established have not been validated [Fakhimi and Validi, 2023]. Despite these theoretical limitations, QAOA has been successfully implemented on quantum hardware to solve small-scale portfolio optimization problems [Stopfer and Wagner, 2025, Herman et al., 2023].

The VQE follows a similar hybrid quantum-classical paradigm but leverages the variational principle to estimate the minimum eigenvalue of the Ising Hamiltonian. In this framework, a Parameterized Quantum Circuit (PQC) prepares a trial wavefunction whose expected energy, computed as the expectation value of the Hamiltonian, is minimized through classical optimization. The VQE algorithm thus yields a solution corresponding to the minimum energy, or optimal portfolio configuration [Peruzzo et al., 2014]. Recent implementations on IBM's superconducting quantum hardware demonstrate the feasibility of this approach on small-scale portfolio instances [Wang et al., 2025a, Herman et al., 2023]. Kolotouros and Wallden [2022] implement both QAOA and VQE for portfolio optimization on simulators, proposing an evolving objective function to improve convergence and nicely visualizing optimization landscapes. Herman et al. [2023] compare QAOA and VQE for constrained portfolio optimization, proposing an approach to include multiple contraints into the problem formulation. Chen et al. [2023] integrate the Black-Litterman model into the quantum domain by using a VQC for modelling investor views, combined with VQE for addressing the asset selection problem. Zaman et al. [2024] systematically benchmark the impact of hyperparameters such as rotation blocks, repetitions, and entanglement types on the performance of QAOA and VQEs for portfolio optimization.

QA, by contrast, realizes the optimization directly on hardware through the adiabatic evolution of a quantum system from an easily prepared ground state toward the ground state of the Ising Hamiltonian. Similarly to the VQE approach, the system's lowest-energy state corresponds to the optimal asset allocation [Kadowaki and Nishimori, 1998, Das and Chakrabarti, 2008]. Tang et al. [2024] compare 14 QA and reverse QA approaches, finding no significant difference in time-to-solution, but an improved success probability with coupler and counter-diabatic QA. Further early demonstrations on D-Wave devices have demonstrate that quantum annealers can efficiently explore large combinatorial search spaces and produce competitive approximate solutions, especially when integrated with classical pre- and post-processing routines [Rosenberg et al., 2016, Venturelli and Kondratyev, 2019, Cohen et al., 2020].

Table 2.1: Overview of quantum portfolio optimization literature (chronological order)

Paper	**Problem Formulation**	**Approach**	**Implementation**	**Classical Benchmarks**	**Dynamic**	**Forward-looking**
Rosenberg et al. [2016]	Static discrete Markowitz (see Eq. 2.5)	QA	QPU			
Venturelli and Kondratyev [2019]	Static binary Markowitz (see Eq. 2.4)	QA	QPU	✓		
Cohen et al. [2020]	Static binary Markowitz (see Eq. 2.4)	QA	QPU	✓		
Phillipson and Bhatia [2021]	Static binary Markowitz (see Eq. 2.4)	QA	QPU	✓		
Mugel et al. [2022]	Static binary Markowitz (see Eq. 2.4)	QA and VQE	QPU	✓	✓	
Kolotouros and Wallden [2022]	Static binary Markowitz (see Eq. 2.4)	QAOA and VQE	Simulator			
Brandhofer et al. [2022]	Static binary Markowitz (see Eq. 2.4)	QAOA	QPU			
Heras et al. [2023]	Static binary Markowitz (see Eq. 2.4)	VQE	QPU	✓		
Herman et al. [2023]	Static continuous Markowitz (see Eq. 2.5)	QAOA and VQE	Simulator and QPU			
Jain et al. [2023]	Static binary Markowitz (see Eq. 2.4)	Hybrid Annealer-Gate	Simulator			
Buonaiuto et al. [2023]	Static continuous Markowitz (see Eq. 2.5)	VQE	QPU	✓		
Chen et al. [2023]	Static binary Black-Litterman (see Eqs. 2.4 & 2.2)	VQC + VQE	Simulator	✓		
Aguilera et al. [2024]	Static discrete Markowitz + CO_2 emission constraints	QA	Simulator and QPU	✓		
Catalano et al. [2024]	Static discrete Markowitz + ESG constraints	QA	QPU			
Quynh et al. [2024]	Static binary Markowitz (see Eq. 2.4)	Extension of VQE/QAOA	Simulator			
Tang et al. [2024]	Static binary Markowitz (see Eq. 2.4)	Various variants of QA	Simulator	✓		
Chou et al. [2024]	Static discrete Markowitz (see Eq. 2.5)	QA	QPU	✓		
Zaman et al. [2024]	Static binary Markowitz (see Eq. 2.4)	QAOA and VQE	Simulator	✓		
Salirrosas et al. [2025]	Static binary Markowitz + market sentiment indicator	VQE	Simulator			
Wang et al. [2025b]	Static binary Markowitz (see Eq. 2.4)	VQE	Simulator and QPU			
Wang et al. [2025a]	Static continuous Markowitz (see Eq. 2.5)	QA	QPU	✓		
Stopfer and Wagner [2025]	Static binary Markowitz (see Eq. 2.4)	QA and QAOA	QPU	✓		
The proposed approach	Dynamic continuous state-action space (see Eq. 3.3)	VQC-RL	Simulator + QPU	✓	✓	✓

Note: In the context of this table, "and" indicates that multiple approaches were implemented and evaluated side by side, whereas "+" signifies that the components were used in conjunction as part of a single approach.

However, practical performance is still shown to be limited by noise, qubit connectivity, and embedding constraints [Phillipson and Bhatia, 2021, Sakuler et al., 2025]. A recent benchmark study by Stopfer and Wagner [2025] sheds further doubt on the practical utility of QA and QAOA for portfolio optimization, showing that classical heuristics can solve small portfolio optimization problems in seconds while outperforming quantum approaches in solution quality, indicating that current hardware does not yet facilitate a quantum advantage in solving complex QUBO formulations.

In addition, Stopfer and Wagner [2025] show in their benchmarking study that modern classical solvers can still exactly solve small portfolio optimization problems (tens of assets) in seconds, and a tailored classical heuristic outperforms quantum approaches in solution quality, indicating that current hardware does not yet facilitate a quantum advantage in portfolio optimization.

It's also important to note, that, similarly to classical MVO, all of the mentioned approaches are confined to static optimization formulations (solving for optimal asset allocation for a given time point). A notable effort tackling dynamic or sequential portfolio decisions using quantum techniques is Mugel et al. [2022], who implemented several quantum and quantum-inspired algorithms for dynamic portfolio optimization on real-world data. They tested D-Wave's hybrid QA, two VQE-based methods on IBM quantum processors, and a quantum-inspired tensor network method. This study was the first to apply quantum algorithms to a multi-step portfolio rebalancing scenario, highlighting both the potential and the challenges of moving beyond static use-cases. However, even in this dynamic study, the quantum approaches essentially solve a static encoding of the multi-period problem (by embedding multiple time steps into one large optimization problem) rather than learning an adaptive trading policy.

In summary, while there has been steady progress in applying quantum annealers and variational quantum algorithms to portfolio optimization, these efforts have so far focused on static optimization or multi-period static formulations, and classical methods still often hold an edge in performance under practical conditions. This leaves open the question of how to leverage quantum computing for truly adaptive, sequential decision-making in finance and whether a quantum advantage can be realized under those conditions.

2.2 Quantum Reinforcement Learning

QRL is a nascent but growing area of research that seeks to merge RL techniques with quantum computing. In broad terms, QRL algorithms use quantum resources to either accelerate RL computations or to represent policy/value functions in new ways [Meyer et al., 2024]. The most actively studied branch of QRL involves VQCs serving as function approximators (replacing classical counterparts in a RL algorithm). In these approaches, the agent interacts with a classical environment as usual, but the policy or Q-value function is encoded by a quantum circuit whose parameters are trained via classical optimization (loss backpropagation). This paradigm fits the capabilities of today's noisy intermediate-scale quantum (NISQ) devices and its feasibility has been demonstrated in standard control environments. For example, hybrid agents using a VQC within a DQN were shown to learn optimal actions in discrete tasks like FrozenLake[1], using significantly fewer parameters than an equivalent classical DQN [Chen et al., 2020]. Similarly, Lockwood and Si [2020] incorporate VQC approximators into value-based frameworks and demonstrate its ability to solve simple continuous control tasks like CartPole[2] and Blackjack, especially when efficient quantum state encoding schemes were employed. These results suggest that quantum models can represent policies or value functions with a compact encoding, offering sample-efficiency and faster convergence in some cases.

Beyond value-based RL, prior work has also explored quantum variants of policy-gradient and Actor-Critic methods. Jerbi et al. [2021] introduce quantum policy iteration and provide theoretical evidence for a quantum advantage in reinforcement learning for specific cases. Lin et al. [2025] integrate VQCs with non-local observables into DQN and Asynchronous Advantage Actor-Critic frameworks. They show that a VQC-enhanced agent can match or exceed the performance of a classical agent, with indications of faster learning thanks to the expressive power of quantum circuits. These works illustrate that QRL algorithms can be realized on current hardware and they hint at possible advantages, for example, using fewer parameters or achieving better sample efficiency. However, QRL is still in an exploratory stage—most demonstrations so far are on small-scale or toy problems, and practical evidence of a quantum advantage in complex, real-world RL tasks remains elusive. A notable application in finance is Cherrat et al. [2023] who apply QRL to solve hedging problems. They implement quantum policy-search and distributional Actor-Critic algorithms for optimal

[1] **FrozenLake:** A benchmark environment from OpenAI Gym that tests discrete-state, discrete-action control. The agent must navigate a grid world (a frozen lake) from a start to a goal location while avoiding holes.

[2] **CartPole:** A control task where the agent must balance a pivoted pole attached to a controllable cart (left/right). The system provides continuous state observations (cart position, velocity, pole angle, and angular velocity).

hedging of financial derivatives (a sequential decision problem), using VQCs in the agent's policy and value functions. Their results show that QRL could achieve performance comparable to classical Deep RL while using significantly fewer trainable parameters, and the quantum model even outperformed classical baselines in certain metrics. Moreover, they successfully ran their QRL agent on a trapped-ion processor with 16 qubits, demonstrating the feasibility of QRL on present-day quantum hardware. Another finance-related example is the work of Yang [2023], who explore a QRL approach for American option pricing, a complex sequential optimization problem without an analytical closed-form solution. By integrating quantum computing into a Deep RL framework for option pricing, they improve the efficiency of learning optimal exercise policies. While these studies underscore the potential of QRL in financial decision problems, portfolio optimization has not yet been addressed with QRL, despite classical RL methods achieving state-of-the-art performance in this domain [Zhang et al., 2020].

2.3 Identified Gaps

There is a lack of frameworks where a quantum or hybrid quantum-classical algorithm learns a policy over time in response to market state changes. In other words, the sequential nature of portfolio rebalancing has been largely absent from quantum approaches. The focus in the quantum domain so far has been on portfolio optimization formulated as a static QUBO, leaving a void in addressing forward-looking dynamic decision-making.

The majority of QRL research has focused on basic control tasks, indicating a need for more development and practical demonstrations of QRL algorithms on complex real-world problems. Existing literature lacks any demonstration of QRL for portfolio optimization, despite the clear success of RL in classical portfolio management and the active interest in quantum methods for static portfolio optimization. The only finance-related QRL works so far have addressed option pricing and hedging.

In this paper, we aim to bridge these gaps by proposing a QRL framework for portfolio optimization and demonstrating its implementation with real-world financial data on quantum hardware.

3 Methodology

3.1 Foundations of Quantum Computing

Quantum computing builds on the principles of quantum mechanics to provide a fundamentally different model of information processing compared to classical digital computation [Nielsen and Chuang, 2010]. Whereas classical computers operate on bits that take values in $\{0, 1\}$, quantum computers manipulate quantum bits, or *qubits*, which are realized physically as controllable two-level quantum systems. Depending on the hardware platform, qubits can be encoded in electron spins, photon polarizations, trapped ions, or current loops in superconducting circuits. All of these implementations share the same mathematical structure: a state described by a vector in a two-dimensional complex Hilbert space.

A single qubit is most generally written as a coherent superposition of the computational basis states $|0\rangle$ and $|1\rangle$,

$$\boldsymbol{\psi} : \ \mathbb{C}^2 \to \mathcal{H}_2 \,, \quad (\alpha, \beta) \ \mapsto \ \alpha|0\rangle + \beta|1\rangle, \quad \text{s.t.} \quad |\alpha|^2 + |\beta|^2 = 1 \,.$$

where $\{|0\rangle, |1\rangle\}$ forms an orthonormal basis of the two-dimensional Hilbert space $\mathcal{H}_2$ and the coefficients $\alpha, \beta \in \mathbb{C}$ are called the probability amplitudes. The normalization condition ensures that a measurement of the qubit in the computational basis yields either $|0\rangle$ or $|1\rangle$ with probabilities $|\alpha|^2$ and $|\beta|^2$, respectively. While this superficially resembles a probabilistic mixture, the state is fundamentally more expressive: the relative phase between α and β enables constructive and destructive interference under unitary transformations of the qubit, a phenomenon with no classical analogue.

The geometry of qubits is most clearly seen through the Bloch sphere representation. Any pure qubit state can be re-parameterized as

$$\boldsymbol{\psi} = \cos\!\left(\tfrac{\theta}{2}\right) \, |0\rangle + e^{i\phi} \sin\!\left(\tfrac{\theta}{2}\right) \, |1\rangle \,.$$

with angles $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$. This maps each qubit state to a unique point on the surface of a unit sphere, called the Bloch sphere. The computational basis states $|0\rangle$ and $|1\rangle$ occupy the north and south poles, respectively, while any other point on the sphere represents a coherent superposition $\alpha|0\rangle + \beta|1\rangle$ with relative phase and amplitude determined by its position (see Figure 3.1).

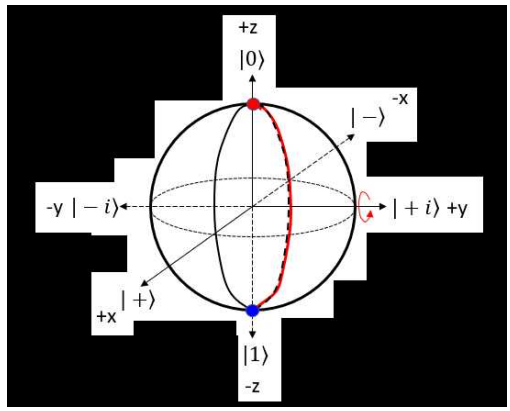

Figure 3.1: Visualization of the Bloch sphere [Freiman, 2024].

From a geometric perspective, quantum gates on a single qubit are rotations of this vector on the Bloch sphere. The abstract rotation operator

$$\mathbf{R}_{\hat{n}}(\theta) = e^{-i\,\frac{\theta}{2}\,\hat{n}\boldsymbol{\sigma}}\,,$$

where $\boldsymbol{\sigma} = (\mathbf{X}, \mathbf{Y}, \mathbf{Z})$ denotes the Pauli matrices, represents a physical rotation of the qubit state by an angle θ about the Bloch-sphere axis $\hat{n}$, implemented physically through controlled electromagnetic pulses.

When multiple qubits are combined, the state space is given by the tensor product of the individual qubit spaces. A system of n qubits is described by a vector in a 2^n-dimensional Hilbert space. For example, a two-qubit state has the general form

$$\boldsymbol{\psi} = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle \quad \text{s.t.} \quad \sum |\alpha_{ij}|^2 = 1\,, \quad \boldsymbol{\psi} \in \mathcal{H}_4\,,$$

The distinguishing feature of such multi-qubit states is not merely their dimensionality, but the presence of complex phases across the amplitudes α_{ij}. These phases give rise to coherent interference when the system is acted upon by unitary transformations. As the number of qubits increases, interference can occur across an exponentially growing set of basis states. This rapidly expanding interference structure is the fundamental source of the expressive power of multi-qubit quantum systems.

However, coherent interference across multiple qubits requires physical interactions between them. In practice, such interactions are realized through two-qubit gates—most notably the Controlled-NOT (CNOT) gate, which conditionally flips the state of the second qubit (the target qubit) if and only if the first qubit (the control qubit) is $|1\rangle$. It thereby creates correlations between qubits that cannot be produced by single-qubit rotations alone. Such correlations between qubits are referred to as entanglement and the gates that facilitate them are called entagling gates [Horodecki et al., 2009].

Quantum gates are designed to be universal, meaning that any unitary transformation on n qubits can be decomposed into a sequence of single-qubit rotations and a fixed entangling gate such as CNOT [Barenco et al., 1995]. On this foundation, several quantum algorithms have been developed that provably outperform their best-known classical counterparts. Examples include Shor's algorithm for factoring large integers in polynomial time [Shor, 1997], Grover's algorithm for unstructured search offering quadratic speedup [Grover, 1997], and quantum simulation algorithms for chemistry and materials science [Aspuru-Guzik et al., 2005]. While portfolio optimization is not yet among the domains with a proven theoretical quantum advantage, the same principles of state space expressivity and interference apply to optimization and learning tasks.

However, realizing quantum computation in practice comes with multiple challenges. Interactions between qubits and their environment lead to the loss of quantum phase information, a phenomenon referred to as decoherence. Qubits must exhibit coherence times long enough to support computationally meaningful operations, while simultaneously remaining accessible to precise external control through electromagnetic pulses. Balancing isolation from the environment with controllability constitutes a central engineering challenge. In addition, all existing quantum devices are subject to noise: unwanted environmental interactions, gate errors caused by control pulses, and measurement errors. This defines the so-called Noisy Intermediate-Scale Quantum (NISQ) era, enabling machines with tens to hundreds of non-error-corrected qubits [Preskill,

2018]. In this regime, fault-tolerant computation is not yet feasible, and algorithm design must explicitly account for noise.

3.2 Variational Quantum Circuits and Quantum Neural Networks

PQCs form the cornerstone of modern quantum machine learning and variational quantum algorithms. A PQC is a quantum circuit with tunable parameters in which a parameterized unitary transformation $\mathbf{U}(\boldsymbol{\theta})$ acts on an input state. VQCs are a subclass of PQCs in which the tunable parameters are additionally optimized according to a cost function defined on measurement outcomes [Benedetti et al., 2019]. This optimization is typically performed using gradient-based methods, where gradients can be estimated using techniques like the parameter-shift rule [Schuld et al., 2019]. When viewed through the lens of machine learning, the parametrization and optimization of these circuits motivates the terminology Quantum Neural Network (QNN) [Cerezo et al., 2021]. However, there are different approaches frequently lumped together under the term QNN, encompassing models as diverse as quantum associative memories, quantum perceptrons, and interacting quantum-dot systems [Schuld et al., 2014]. We therefore avoid this ambiguous terminology and will henceforth refer to VQCs.

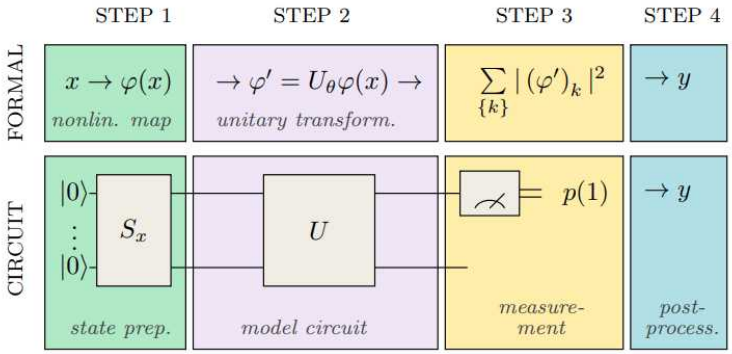

Figure 3.2: Illustration of the four steps involved in inference with a VQC [Adapted from Schuld et al., 2020].

As illustrated in Fig. 3.2, the general workflow of a VQC consists of four fundamental steps: (i) *state preparation*, where classical input data $\mathbf{x}$ is encoded into a quantum state, (ii) *model transformation*, where the parametrized unitary $\mathbf{U}(\boldsymbol{\theta})$ acts on the encoded state, (iii) *measurement*, where expectation values of observables are sampled from the output state, and (iv) *post-processing*, where the measurement outcomes are, for example, used to compute a cost function.

In the first step, classical data must be embedded onto a quantum state. There are multiple strategies for this data encoding, and the choice of method strongly influences both the expressive capacity of the circuit and its resource requirements. Three common approaches are basis encoding, angle encoding, and amplitude encoding [Schuld et al., 2021].

In **basis encoding**, the simplest type of classical data representation, each component of a binary string $\mathbf{x} \in \{0,1\}^n$ is directly mapped to one of the (discrete) computational basis states of an n-qubit register:

$$\{0,1\}^n \to \mathcal{H}_{2^n}\,, \quad \mathbf{x} \mapsto |\mathbf{x}\rangle = |x_1 x_2 \cdots x_n\rangle\ .$$

This approach provides an intuitive one-to-one correspondence between classical bit strings and quantum states, enabling straightforward representation of discrete variables. However, it requires that all input features be represented in binary form prior to encoding. Moreover, basis encoding consumes one qubit per binary feature, limiting scalability when dealing with high-dimensional or real-valued datasets [Nielsen and Chuang, 2010].

Angle encoding, also known as rotation encoding, represents classical data by assigning each component of a real-valued vector $\mathbf{x} \in \mathbb{R}^n$ to the rotation angle of a parameterized quantum gate, e.g. $\mathbf{R}_Y$, typically acting on a qubit initialized in the $|0\rangle$ state:

$$\mathbb{R}^n \to \mathcal{H}_{2^n}\,, \quad \mathbf{x} \mapsto \left(\bigotimes_{j=1}^{n} \mathbf{R}_Y(x_j) \right) |0\rangle^{\otimes n}\,,$$

where $\mathbf{R}_Y(x_j)$ denotes a rotation about the y-axis of the Bloch sphere by an angle proportional to the feature value x_j. This maps continuous input data directly onto the Bloch sphere, with each qubit encoding one feature as a distinct quantum rotation [Havlíček et al., 2019].

Angle encoding offers a hardware-efficient way to represent continuous data and is widely used in VQCs due to its low resource requirements. However, since each feature requires its own qubit and rotation gate, the representational capacity is linear in the number of available qubits, limiting its scalability compared to amplitude encoding.

Amplitude encoding is a data representation scheme that embeds the components of a classical vector into the amplitudes of a quantum state. The idea dates back to early quantum algorithms such as the Harrow-Hassidim-Lloyd algorithm for solving linear systems [Harrow et al., 2009], and was formalized as a state-preparation problem in Möttönen et al. [2004] and Plesch and Brukner [2011]. In this scheme, the components of a classical feature vector are mapped directly to the probability amplitudes of the quantum system's basis states. Each amplitude corresponds to one possible realization of the quantum system, so that all information about the input vector is distributed across the superposition of states rather than stored in a single register element. Since a quantum wavefunction represents a probability distribution over all possible basis states, its squared amplitudes must collectively sum to one, i.e. $\sum_i |x_i|^2 = 1$. To enforce this normalization condition for arbitrary real-valued vectors, the input is divided by its ℓ_2-norm, ensuring that the resulting quantum state has unit total probability.

To illustrate, consider a two-qubit system: it spans a four-dimensional Hilbert space with computational basis states $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$. Given an arbitrary real vector $\mathbf{x} \in \mathbb{R}^4$, its amplitude-encoded quantum representation is

$$|\boldsymbol{\psi}(\mathbf{x})\rangle = \frac{1}{\|\mathbf{x}\|_{\ell_2}} \big(x_1|00\rangle + x_2|01\rangle + x_3|10\rangle + x_4|11\rangle\big)\,, \quad \text{where} \quad \|\mathbf{x}\|_{\ell_2}^2 = \sum_{i=1}^{4} x_i^2\ .$$

This means that the quantum system simultaneously encodes all entries of the classical vector as part of a coherent superposition, where the probability of measuring each basis state equals the squared magnitude of its corresponding amplitude. The advantage of this encoding is that a single n-qubit register can represent 2^n classical features in parallel, in this case 4 data points with only 2 qubits. Amplitude encoding hence provides an exponentially compact data embedding and enables efficient quantum state manipulations.

Formally, any input vector $\mathbf{x} \in \mathbb{R}^{2^n}$ can be amplitude-encoded on an n-qubit quantum state as defined by

$$\mathbb{R}^{2^n}_{\setminus\{\mathbf{0}\}} \to \mathcal{H}_{2^n}\,, \quad \mathbf{x} \mapsto \frac{1}{\|\mathbf{x}\|_{\ell_2}} \sum_{i=1}^{2^n} x_i\,|i\rangle, \quad \|\mathbf{x}\|_{\ell_2}^2 = \sum_{i=1}^{2^n} x_i^2\ . \tag{3.1}$$

To translate this mathematical idea into an actual quantum circuit, Möttönen et al. [2004] propose a state-preparation algorithm that constructs a PQC capable of encoding any such amplitude-encoded quantum state. Their method decomposes the target state into a sequence of uniformly controlled rotations that iteratively align the amplitudes of computational basis states with the desired vector components. The circuit is constructed through a recursive sequence of uniformly controlled rotations—typically implemented with $\mathbf{R}_Y$ and $\mathbf{R}_Z$ gates for real and complex amplitudes, respectively, or more generally with arbitrary single-qubit unitaries $\mathbf{U}(\theta, \phi, \lambda)$—and conditional phase shifts, such that each rotation step adjusts the relative weights of the remaining amplitudes while preserving normalization. The number of single- and two-qubit gates in the resulting circuit scales with $\mathcal{O}(2^n)$ for an n-qubit register and achieves amplitude encoding of $\mathbf{x}$ onto the quantum state $|\boldsymbol{\psi}(\mathbf{x})\rangle$.

After encoding the classical data as a quantum state, in the second step of inference with a VQC the parametrized unitary $\mathbf{U}(\boldsymbol{\theta})$ is constructed from alternating layers of single-qubit rotations and multi-qubit entangling gates. Single-qubit rotations, such as $\mathbf{R}_X(\theta)$, $\mathbf{R}_Y(\theta)$, and $\mathbf{R}_Z(\theta)$, rotate individual qubits about the x, y, or z axes of the Bloch sphere by angles determined by the corresponding parameters $\theta \in \boldsymbol{\theta}$.

These unitary gates allow the circuit to explore arbitrary directions within the Hilbert space of each qubit. Entangling layers, typically implemented with CNOT gates, couple the qubits and create non-classical correlations across them. By stacking multiple layers of such rotation-entanglement blocks, PQCs can approximate increasingly expressive functions, analogous to the role of deeper layers in classical neural

networks. Figure 3.3 shows an example of a simple PQC with three qubits, three layers of rotations, two layers of entangling gates, and three measurement operations.

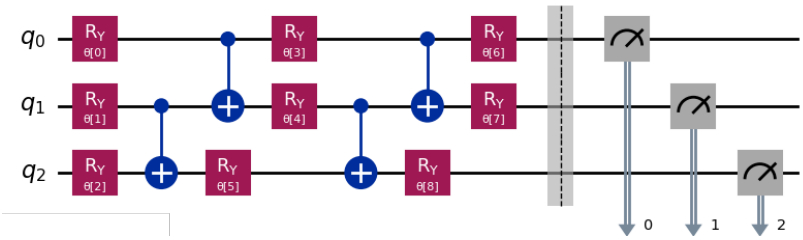

Figure 3.3: A PQC with three qubits, nine parameterized single-qubit rotation gates (red) and four two-qubit entangling CNOT gates (blue).

Once we construct the PQC $\mathbf{U}(\boldsymbol{\theta})$ which includes input encoding and parameterized transformation, the final step is to extract the information from it. Generally, measurements in variational quantum algorithms are defined by Hermitian observables $\widehat{\boldsymbol{B}}$, whose expectation values encode task-relevant information about the quantum state. Given an observable $\widehat{\boldsymbol{B}}$ and input state $|\boldsymbol{x}\rangle$, a quantum circuit defines the expectation value:

$$f(\boldsymbol{x};\boldsymbol{\theta}) = \langle \boldsymbol{x}|\, \mathbf{U}^{\dagger}(\boldsymbol{\theta})\, \widehat{\boldsymbol{B}}\, \mathbf{U}(\boldsymbol{\theta})\, |\boldsymbol{x}\rangle.$$

Common choices for observables include Pauli operators and their tensor products, among which, the Pauli-Z operator is particularly convenient, as it corresponds to measurement in the computational basis and directly quantifies the position on the z-axis of the Bloch sphere, that is, the population imbalance between $|0\rangle$ and $|1\rangle$. This makes Pauli-Z expectation values naturally interpretable as the probability of measuring $|1\rangle$, and they are therefore widely used as readout observables in variational quantum circuits.

However, quantum measurements are inherently probabilistic: measuring a qubit yields a binary outcome, corresponding to a collapse onto either $|0\rangle$ or $|1\rangle$. As a result, expectation values cannot be obtained from a single circuit execution, but must instead be estimated empirically by repeating the circuit multiple times (shots). The repeated sampling of the PQC yields an estimate of the probability distribution over measurement outcomes. The expectation values of specific observables (averages of the observed outcomes) constitute the model outputs.

In the context of variational algorithms, these measured quantities are then used to evaluate a real-valued cost function $C(\boldsymbol{\theta})$ that quantifies the deviation between the model predictions and the desired target values. The parameters $\boldsymbol{\theta}$ are updated based on this cost, forming a closed-loop optimization process akin to classical neural networks.

This optimization requires gradients of expectation values with respect to the parameters $\boldsymbol{\theta}$. Mitarai et al. [2018] demonstrate that, for parametrized quantum gates of the form $\mathbf{U}(\theta) = e^{-i\,\theta\mathbf{G}/2}$ where $\mathbf{G}$ is Hermitian with two distinct eigenvalues, the gradient of an observable expectation value can be evaluated analytically via what is now known as the *parameter-shift rule*:

The derivative of f with respect to a single parameter is given by

$$\frac{\partial}{\partial\theta_i} f(\boldsymbol{x};\boldsymbol{\theta}) = \tfrac{1}{2}\left[f\big(\boldsymbol{x};\boldsymbol{\theta}+\tfrac{\pi}{2}\hat{e}_i\big) - f\big(\boldsymbol{x};\boldsymbol{\theta}-\tfrac{\pi}{2}\hat{e}_i\big)\right], \tag{3.2}$$

where $\hat{e}_i$ denotes the unit vector in the ith coordinate direction.

Since this condition holds for single-qubit rotation gates such as $\mathbf{R}_X(\theta)$, $\mathbf{R}_Y(\theta)$, and $\mathbf{R}_Z(\theta)$, the rule can be directly applied to the trainable layers of a VQC to compute exact gradients of the circuit output with respect to its parameters. In other words, the gradient of the expectation value can be calculated by evaluating the same VQC twice per parameter, with the i-th parameter shifted by $\pm\frac{\pi}{2}$ while keeping all others fixed. This enables unbiased gradient estimation directly from quantum measurements [Schuld et al., 2019].

By iteratively adjusting the parameter vector $\boldsymbol{\theta}$ to minimize the cost function $C(\boldsymbol{\theta})$, the VQC converges to a high-order Fourier series representation of the provided input data, as illustrated in Figure 3.4. It therefore acts as an *asymptotically universal* approximator.

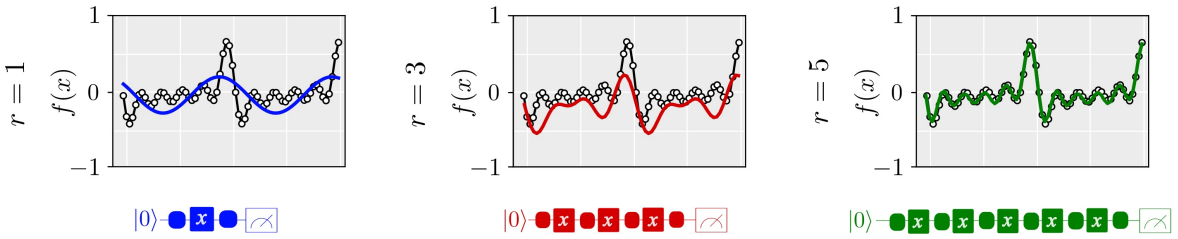

Figure 3.4: Visualization of converged VQCs with one, three, and five layers of rotation and entanglement gates, illustrating how increasing circuit depth enables the representation as higher-order Fourier series. [Adapted from Schuld et al., 2021].

The expressive power of VQCs lies in their ability to generate highly entangled states over exponentially large Hilbert spaces, but practical training suffers from the phenomenon of barren plateaus, in which gradients vanish exponentially with system size and circuit depth [McClean et al., 2018]. This imposes strong constraints on circuit design, initialization, and ansatz selection, motivating problem-inspired architectures and shallow circuits adapted to the NISQ regime, like the one we propose in this work.

3.3 Reinforcement Learning

Reinforcement Learning (RL) is a computational framework for learning behaviour through trial-and-error interaction with a dynamic environment. At each discrete time step t, the agent observes a state $\mathbf{s}_t$, selects an action $\mathbf{a}_t$, receives a reward r_t, and transitions to a new state $\mathbf{s}_{t+1} \sim P(\cdot \mid \mathbf{s}_t, \mathbf{a}_t)$. The probability of taking action $\mathbf{a}$ in state $\mathbf{s}$ is defined by the agent's policy $\pi(\mathbf{a} \mid \mathbf{s})$. The optimal policy π^* is defined as the one that maximizes the Bellman equations $V^\pi(\mathbf{s})$ and $Q^\pi(\mathbf{s}, \mathbf{a})$ for the entire state-action space [Bellman, 1957].

$$V^\pi(\mathbf{s}) = \mathbb{E}_{\mathbf{a}\sim\pi(\cdot|\mathbf{s})}\left[R(\mathbf{s},\mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}'\sim P(\cdot|\mathbf{s},\mathbf{a})} V^\pi(\mathbf{s}')\right],$$

$$Q^\pi(\mathbf{s},\mathbf{a}) = R(\mathbf{s},\mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}'\sim P(\cdot|\mathbf{s},\mathbf{a})} \mathbb{E}_{\mathbf{a}'\sim\pi(\cdot|\mathbf{s}')} Q^\pi(\mathbf{s}',\mathbf{a}').$$

While the Bellman equations characterize optimal behaviour, they do not by themselves provide a practical solution in most settings. Solving them exactly requires either full knowledge of the environment dynamics or exhaustive enumeration of the state-action space, both of which quickly become intractable as problem complexity grows. RL algorithms can be viewed as different strategies for approximately solving one of the Bellman optimality equations under varying assumptions.

We summarize the algorithms most relevant for contextualizing our methodological framework, but there exists a wide range of approaches to solve this problem. For a comprehensive treatment of RL and its origins, we refer the reader to Sutton and Barto [2015].

Q-Learning and Deep Q-Networks

Q-learning [Watkins and Dayan, 1992] is an off-policy Temporal-Difference (TD) control algorithm that learns the optimal action-value function independently of the behaviour policy. Unlike on-policy methods, Q-learning updates action values toward the maximum action-value of the next state, regardless of which action is actually taken, as formalized in Algorithm 1. This property allows Q-learning to learn optimal policies even when following exploratory or suboptimal behaviour policies, making it a powerful and widely used RL algorithm.

Algorithm 1 Q-Learning

Initialize action-value function $Q(\mathbf{s}, \mathbf{a})$ arbitrarily
for each episode **do**
  Initialize $\mathbf{s}$
  repeat
    Select $\mathbf{a}$ via ε-greedy w.r.t. $Q(\mathbf{s}, \cdot)$
    Execute $\mathbf{a}$, observe r, $\mathbf{s}'$
    $Q(\mathbf{s}, \mathbf{a}) \leftarrow Q(\mathbf{s}, \mathbf{a}) + \alpha \left[r + \gamma \max_{\mathbf{a}'} Q(\mathbf{s}', \mathbf{a}') - Q(\mathbf{s}, \mathbf{a}) \right]$
    $\mathbf{s} \leftarrow \mathbf{s}'$
  until terminal condition
end for

Action selection is implemented using the ε-greedy policy: with probability $1 - \varepsilon$, the agent selects the action with the highest estimated action value, while with probability ε it selects an action uniformly at random. This simple exploration strategy ensures sufficient state-action space coverage while maintaining a bias toward actions that are currently estimated to be optimal. While tabular Q-learning converges under suitable conditions, it also becomes computationally infeasible in large state-action spaces where the action-value function cannot be stored explicitly.

DQNs [Mnih et al., 2015] address this limitation by approximating $Q(s, a)$ with a neural network. To stabilize training, DQNs employ experience replay and a separate target network, which may be updated periodically or via soft updates. The resulting training procedure is delineated in Algorithm 2. These extensions enable Q-learning to scale to high-dimensional observation spaces and form the foundation of modern value-based Deep RL.

However, the reliance on an explicit maximization over actions (see Line 12 of Algorithm 2) still restricts DQNs to discrete action spaces. In continuous settings, this maximization can only be approximated by sampling a finite set of candidate actions and selecting the best among them. This limitation motivates Actor-Critic methods, which learn explicit policies and enable direct action sampling together with more flexible exploration mechanisms.

Algorithm 2 Deep Q-Network (DQN)

1: Initialize action-value network $Q(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q})$ arbitrarily
2: Initialize target network $Q'(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q'}) \leftarrow Q(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q})$
3: Initialize replay buffer $\mathcal{D}$
4: Initialize target network update period C
5: **for** each episode **do**
6: Receive initial state $\mathbf{s}$
7: **for** each step **do**
8: Select action $\mathbf{a}$ via ε-greedy w.r.t. $Q(\mathbf{s}, \cdot)$
9: Execute $\mathbf{a}$, observe reward r and next state $\mathbf{s}'$
10: Store transition $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}')$ in $\mathcal{D}$
11: Sample minibatch $\{(\mathbf{s}_j, \mathbf{a}_j, r_j, \mathbf{s}'_j)\}_{j=1}^{N} \sim \mathcal{D}$
12: Compute target: $y_j \leftarrow r_j + \gamma \max_{\mathbf{a}'} Q'(\mathbf{s}'_j, \mathbf{a}' \mid \boldsymbol{\theta}^{Q'})$
13: Update $\boldsymbol{\theta}^{\boldsymbol{Q}}$ to minimize MSE loss: $\mathcal{L}(\boldsymbol{\theta}^{\boldsymbol{Q}}) = \frac{1}{N} \sum_{j}^{N} \left(Q(\mathbf{s}_j, \mathbf{a}_j \mid \boldsymbol{\theta}^{Q}) - y_j \right)^2$
14: **if** step mod $C = 0$ **then**
15: Update target network: $\boldsymbol{\theta}^{Q'} \leftarrow \boldsymbol{\theta}^{Q}$
16: **end if**
17: Set $\mathbf{s} \leftarrow \mathbf{s}'$
18: **end for**
19: **end for**

Deep Deterministic Policy Gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG) [Lillicrap et al., 2019] extends deterministic policy gradient methods to deep neural networks and continuous action spaces. In contrast to stochastic Actor-Critic

algorithms, DDPG employs a deterministic actor $\mu(\mathbf{s} \mid \boldsymbol{\theta}^{\mu})$ that maps each state directly to a continuous action, together with a critic $Q(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q})$ that estimates the corresponding action-value function. Training is performed off-policy, leveraging experience replay, target networks, and soft updates to stabilize learning.

The target value y is computed through the target networks $\mu'(\mathbf{s}' \mid \boldsymbol{\theta}^{\mu'})$ and $Q'(\mathbf{s}', \mathbf{a}' \mid \boldsymbol{\theta}^{Q'})$:

$$y = r + \gamma Q'(\mathbf{s}', \mu'(\mathbf{s}' \mid \boldsymbol{\theta}^{\mu'}) \mid \boldsymbol{\theta}^{Q'}),$$

and the critic parameters are updated by minimizing the mean-squared error (MSE) loss

$$\mathcal{L}(\boldsymbol{\theta}^{Q}) = \mathbb{E}_{(\mathbf{s},\mathbf{a},r,\mathbf{s}')\sim\mathcal{D}}\big[\big(Q(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q}) - y\big)^2\big],$$

where $\mathcal{D}$ denotes the replay buffer.

The actor is updated using the deterministic policy gradient, which propagates gradients through the critic with respect to the action. This update adjusts the actor parameters to maximize the critic's estimated action value:

$$\nabla_{\theta^{\mu}} J \approx \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim\mathcal{D}}\left[\nabla_a Q(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q})\big|_{\mathbf{a}=\mu(\mathbf{s})} \nabla_{\boldsymbol{\theta}^{\mu}} \mu(\mathbf{s})\right].$$

Both the actor and the critic are trained using minibatches sampled from $\mathcal{D}$, and the target networks are updated via soft updates. The complete DDPG procedure is outlined in Algorithm 3. Together, these design choices enable stable and efficient learning in high-dimensional continuous control problems.

Algorithm 3 DDPG

Initialize actor $\mu(\mathbf{s} \mid \boldsymbol{\theta}^{\mu})$ and critic $Q(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q})$ network parameters arbitrarily
Initialize target network parameters $\boldsymbol{\theta}^{\mu'} \leftarrow \boldsymbol{\theta}^{\mu}$, $\boldsymbol{\theta}^{Q'} \leftarrow \boldsymbol{\theta}^{Q}$
Initialize replay buffer $\mathcal{D}$
for each episode **do**
    Observe initial state $\mathbf{s}$
    repeat
        Select action $\mathbf{a} \leftarrow \mu(\mathbf{s}) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$
        Execute $\mathbf{a}$, observe reward r and next state $\mathbf{s}'$
        Store transition $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}')$ in $\mathcal{D}$
        Sample minibatch $\{(\mathbf{s}_j, \mathbf{a}_j, r_j, \mathbf{s}'_j)\}_{j=1}^{N} \sim \mathcal{D}$
        Compute target $y = r + \gamma Q'(\mathbf{s}', \mu'(\mathbf{s}' \mid \boldsymbol{\theta}^{\mu'}) \mid \boldsymbol{\theta}^{Q'})$
        Update critic by minimizing $\big(Q(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q}) - y\big)^2$
        Update actor: $\nabla_{\boldsymbol{\theta}^{\mu}} J \approx \frac{1}{N}\sum_j^N \nabla_a Q(\mathbf{s}_j, \mathbf{a} \mid \boldsymbol{\theta}^{Q})\big|_{\mathbf{a}=\mu(\mathbf{s}_j)} \nabla_{\boldsymbol{\theta}^{\mu}} \mu(\mathbf{s}_j)$
        Soft update target networks: $\boldsymbol{\theta}^{Q'} \leftarrow \tau\boldsymbol{\theta}^{Q} + (1-\tau)\boldsymbol{\theta}^{Q'}$, $\boldsymbol{\theta}^{\mu'} \leftarrow \tau\boldsymbol{\theta}^{\mu} + (1-\tau)\boldsymbol{\theta}^{\mu'}$
        $\mathbf{s} \leftarrow \mathbf{s}'$
    until terminal condition
end for

3.4 Reinforcement Learning for Portfolio Optimization

Portfolio optimization is inherently a sequential decision-making problem. Investment decisions are not made in isolation at a single point in time, but rather as part of a dynamic process in which the portfolio is rebalanced within certain time intervals. Market conditions evolve over time, rewards are often delayed, and actions taken today constrain or enable actions tomorrow. These characteristics are difficult to capture with static optimization techniques that treat portfolio construction as a one-shot problem.

RL provides a suitable framework for addressing these issues by casting portfolio management as a sequential control problem, incorporating delayed rewards, state transitions, and the exploration-exploitation tradeoff. Formally, we express portfolio optimization as a dynamic decision-making problem over a state-action space, in which the agent learns a trading policy that maps market states to portfolio allocations in order to maximize a risk-adjusted performance objective over time.

$$
\begin{aligned}
\text{State:} \quad & \mathbf{s}_t = \big[\, \mathbf{p}_{t-L:t},\ \hat{\mathbf{p}}_{t+1:t+F+1} \,\big], \quad \text{Action:} \quad \mathbf{a}_t = \mathbf{w}_t\,, \\
\text{Reward:} \quad & r_t = \mathbf{w}_t^\top \boldsymbol{\mu}_{t-L:t+F+1} - \eta\, \mathbf{w}_t^\top \boldsymbol{\Sigma}_{t-L:t+F+1}\, \mathbf{w}_t\,, \\
\text{Objective:} \quad & \max_{\mathbf{a}_t}\ Q_t(\mathbf{s}_t, \mathbf{a}_t) = \Big[r_t(\mathbf{s}_t, \mathbf{a}_t) + \gamma\, \mathbb{E}\big[Q_{t+1}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) \big] \Big], \\
\text{where} \quad & \mathbf{w}_t \in \mathbb{R}^N : \text{ portfolio weight vector (allowing for short-selling)}, \\
& \mathbf{p}_t \in \mathbb{R}^N : \text{ vector of past asset prices}, \\
& \hat{\mathbf{p}}_t \in \mathbb{R}^N : \text{ vector of forecasted asset prices}, \\
& \eta,\, \gamma \in [0,1] : \text{ risk preference parameter and Bellman equation discount factor}, \\
& L,\, F \in \mathbb{N} : \text{ lookback and forecast window size in days}, \\
& N,\, t \in \mathbb{N} : \text{ number of assets and timestep (day)}.
\end{aligned}
\tag{3.3}
$$

The action corresponds to the portfolio allocation vector at each time step, represented as continuous weights across all assets. The state is constructed from a rolling window of the past daily prices for each asset, concatenated with a forecast, thereby capturing both historical dynamics and short-term predictive information. The reward balances profitability and risk through a combination of average returns and risk-adjusted volatility with a tunable risk preference parameter η, mirroring the quadratic optimization problem.

This formulation integrates risk directly into the reward signal, avoiding the division by risk inherent in Sharpe ratio-based objectives, which can induce uneven loss landscapes and numerical instabilities during training.

Value-based methods such as Q-learning and DQNs rely exclusively on approximating value functions and derive policies indirectly through action-value maximization. While conceptually simple, these approaches become impractical in continuous action spaces without additional approximations, such as action discretization or sampling-based maximization. Moreover, even accurate value-function approximation does not guarantee that the induced policy is near-optimal when function approximation errors are present.

Policy gradient methods take the opposite approach by directly optimizing a parameterized policy. This allows them to operate naturally in continuous action spaces and optimize the control objective explicitly. However, because policy gradients are estimated from sampled trajectories, they often suffer from high variance and strong sensitivity to hyperparameters. In addition, successive gradient estimates are typically computed independently, limiting the accumulation and reuse of information across iterations.

Actor-Critic architectures combine these two perspectives by coupling a policy (the actor) with a value function estimator (the critic). The critic provides a learned baseline that reduces the variance of policy gradient updates, while the actor ensures direct optimization in policy space. This interaction enables more stable learning and improved scalability to high-dimensional state-action spaces, making Actor-Critic methods particularly well suited to financial decision-making problems like portfolio optimization [Konda and Tsitsiklis, 1999].

We therefore adopt DDPG and a continuous-action variant of a DQN with a parameterized actor as the basis for our RL approach. Both methods are designed for continuous action domains such as portfolio optimization, where actions correspond to fractional asset allocations. The key difference between the two algorithms is, that DDPG constructs its targets using the next action proposed by the learned actor, while the continuous DQN instead approximates the action-value maximization by sampling a finite set of possible next actions from the learned actor and selecting the action-value maximizing action. Both approaches are trained off-policy, allowing extensive reuse of historical experience through replay buffers and stabilizing updates via target networks and soft parameter updates, facilitating data efficiency and robust learning.

The complete training procedures are provided in Algorithm 4, which details initialization, replay buffer usage, policy updates, and the respective target computation strategies.

Algorithm 4 DDPG/DQN for Portfolio Optimization

Initialize actor $\mu(\mathbf{s} \mid \boldsymbol{\theta}^{\mu})$ and critic $Q(\mathbf{s}, \mathbf{a} \mid \boldsymbol{\theta}^{Q})$ parameters
Initialize target network parameters: $\boldsymbol{\theta}^{\mu'} \leftarrow \boldsymbol{\theta}^{\mu}$, $\boldsymbol{\theta}^{Q'} \leftarrow \boldsymbol{\theta}^{Q}$
Initialize replay buffer $\mathcal{D}$
for episode $= 1$ to M **do**
  Receive initial state $\mathbf{s}$
  for $t = 1$ to T **do**
    Select action: $\mathbf{a} \leftarrow \mu(\mathbf{s} \mid \boldsymbol{\theta}^{\mu}) + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$
    Execute $\mathbf{a}$, observe reward r and next state $\mathbf{s}'$
    Store transition $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}')$ in $\mathcal{D}$
    Sample minibatch $\{(\mathbf{s}_j, \mathbf{a}_j, r_j, \mathbf{s}'_j)\}_{j=1}^{N} \sim \mathcal{D}$
    if DDPG **then**
      Compute target: $y_j \leftarrow r_j + \gamma Q'(\mathbf{s}'_j, \mu'(\mathbf{s}'_j) \mid \boldsymbol{\theta}^{Q'})$
    else if DQN **then**
      Sample $\{\mathbf{a}'_{j,k}\}_{k=1}^{M} \sim \text{Uniform}(\mathcal{A})$
      Compute target: $y_j \leftarrow r_j + \gamma \max_k Q'(\mathbf{s}'_j, \mathbf{a}'_{j,k} \mid \boldsymbol{\theta}^{Q'})$
    end if
    Update critic: minimize $\mathcal{L}(\boldsymbol{\theta}^{Q}) = \frac{1}{N} \sum_j^N (Q(\mathbf{s}_j, \mathbf{a}_j \mid \boldsymbol{\theta}^{Q}) - y_j)^2$
    Update actor: $\nabla_{\boldsymbol{\theta}^{\mu}} J \approx \frac{1}{N} \sum_j^N \nabla_a Q(\mathbf{s}_j, \mathbf{a} \mid \boldsymbol{\theta}^{Q})\big|_{\mathbf{a}=\mu(\mathbf{s}_j)} \cdot \nabla_{\boldsymbol{\theta}^{\mu}} \mu(\mathbf{s}_j)$
    Soft update target networks: $\boldsymbol{\theta}^{Q'} \leftarrow \tau \boldsymbol{\theta}^{Q} + (1-\tau)\boldsymbol{\theta}^{Q'}, \quad \boldsymbol{\theta}^{\mu'} \leftarrow \tau \boldsymbol{\theta}^{\mu} + (1-\tau)\boldsymbol{\theta}^{\mu'}$
    Set $\mathbf{s} \leftarrow \mathbf{s}'$
  end for
end for

In the following section, we detail how we translate the above classical algorithm into a fully quantum framework based on VQCs, allowing for a controlled comparison between classical and quantum RL in the context of the portfolio optimization problem.

3.5 Quantum Reinforcement Learning for Portfolio Optimization

We adapt two canonical reinforcement learning algorithms to a fully quantum setting: DDPG and Deep Q-Learning. By unifying both classical and quantum approaches within the same RL frameworks, we enable a systematic comparison of their effectiveness for portfolio optimization while demonstrating the flexibility of VQCs in supporting distinct reinforcement learning methodologies. Both methods are implemented in continuous action space, where the action corresponds to a portfolio allocation vector, and both employ an experience replay buffer and target networks to stabilize learning. The architectures are structurally similar, with actor and critic networks realized as PQCs, and training driven by the same interaction loop between agent and environment as described in Algorithm 4 in the previous section. The outputs of the circuits defined as the expectation values of observables $\widehat{\boldsymbol{B}}$ acting on the N first qubits.

$$f_{\text{VQC}}(\mathbf{x}; \boldsymbol{\theta}) = \langle \mathbf{x} | \mathbf{U}^{\dagger}(\boldsymbol{\theta})\, \widehat{\boldsymbol{B}}\, \mathbf{U}(\boldsymbol{\theta}) | \mathbf{x} \rangle \tag{3.4}$$

We map classical financial observations onto the quantum domain by constructing a normalized feature map and encoding it as a quantum state via amplitude encoding. To this end, we extend the canonical amplitude encoding defined in Equation 3.1 by constructing a normalized feature map that transforms the input vector before embedding. Instead of directly encoding the normalized input $\tilde{\mathbf{x}}$, we transform it into a higher-dimensional feature vector $\mathbf{Z}(\tilde{\mathbf{x}})$ that captures both linear and nonlinear dependencies among components of the input. The resulting amplitude-encoded quantum state is defined as

$$|\boldsymbol{\psi}(\mathbf{Z})\rangle = \frac{1}{\|\mathbf{Z}\|_{\ell_2}} \sum_{i=1}^{d} z_i \, |i\rangle, \quad d = \dim(\mathcal{H}),$$

$$\text{where} \quad \mathbf{Z}(\tilde{\mathbf{x}}) = \left[\, \tilde{\mathbf{x}},\ \tilde{\mathbf{x}}^{\odot 2},\ \sin(\tilde{\mathbf{x}}),\ \cos(\tilde{\mathbf{x}}) \,\right],$$

$$\text{and} \quad \tilde{\mathbf{x}} = \frac{\mathbf{x} - \frac{1}{n}\sum_{i=1}^{n} x_i}{\sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(x_i - \frac{1}{n}\sum_{j=1}^{n} x_j\right)^2}} \,.$$

By applying amplitude encoding to the extended feature map $\mathbf{Z}$ rather than to the normalized input $\tilde{\mathbf{x}}$, we break the radial symmetry in the input space caused by the division by the norm. This enhances expressivity and helps to mitigate barren plateaus during training, as the circuit can more easily differentiate between distinct input states.

To extract a portfolio allocation from the actor circuit, we adopt a measurement approach aligned with the asset dimension. Given a state–action space with N assets, we measure the first N qubits of the circuit in the computational basis using repeated shots, yielding empirical marginal probabilities $\mathbf{p} \in [0,1]^N$ for observing the state $|1\rangle$ on each qubit. These probabilities are obtained as expectation values of single-qubit Pauli-Z observables. To obtain a valid portfolio allocation, the resulting vector is ℓ_1-normalized to produce portfolio weights $\mathbf{w} \in \mathbb{R}^N$ satisfying $\sum_i w_i = 1$. This normalization enforces full capital allocation while allowing both long and short positions, with positive and negative weights corresponding to long and short exposures, respectively.

$$\mathbf{w}(\mathbf{x};\boldsymbol{\theta}) = \mathcal{N}\Big(\langle \mathbf{x}|\mathbf{U}^\dagger(\boldsymbol{\theta})\,\widehat{\mathbf{B}}\,\mathbf{U}(\boldsymbol{\theta})|\mathbf{x}\rangle\Big)\,, \tag{3.5}$$

$$\text{where} \qquad \widehat{\mathbf{B}} = (Z_1, \ldots, Z_N)\,, \qquad \mathcal{N}(\mathbf{z})_i = \frac{z_i}{\sum_{j=1}^{N} z_j}, \quad i = 1, \ldots, N\,.$$

To compute gradients of expectation values with respect to circuit parameters, we employ the parameter-shift rule described in Equation 3.2. This technique enables exact and unbiased gradient estimation for variational circuits composed of parameterized rotation gates, without requiring analytical differentiation or finite-difference approximations. Each parameter update involves two forward evaluations of the circuit with fixed parameter shifts, yielding an exact derivative for every trainable parameter θ_i.

The resulting gradients $\nabla_{\boldsymbol{\theta}} f_{\mathrm{VQC}}(\mathbf{x};\boldsymbol{\theta})$ are fully compatible with classical optimization routines. They can be directly integrated into standard backpropagation pipelines, where the quantum circuit is treated as a differentiable module within the computational graph. This allows the use of conventional gradient-based optimizers such as stochastic gradient descent or Adam [Kingma and Ba, 2017] to iteratively minimize a real-valued loss function defined on the quantum circuit outputs.

Unlike hybrid models that retain a classical function approximator (e.g. a neural network) side by side with the VQC, our formulation is *fully quantum*: both actor and critic are represented directly in Hilbert space. Only the calculation of the loss and backpropagation are conducted on classical hardware. This architecture provides a quantum-native instantiation of reinforcement learning, allowing us to assess the representational benefits of variational circuits while keeping the training dynamics comparable to classical baselines. Figure 3.5 depicts this architecture schematically for the illustrative case of two assets and two historical timesteps.

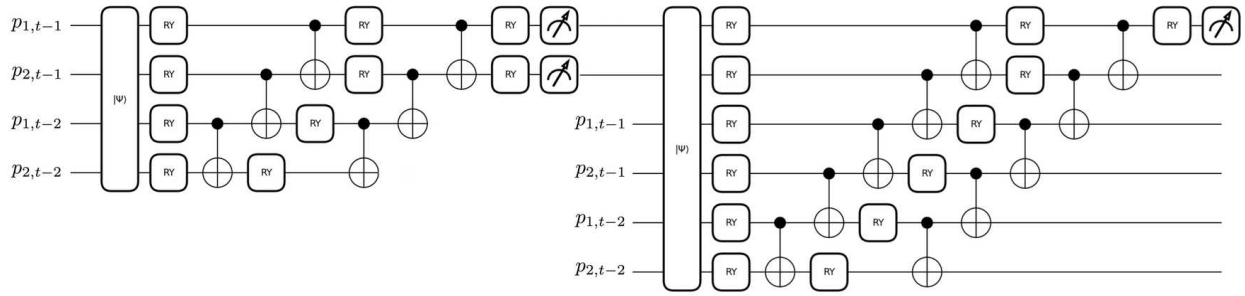

Figure 3.5: Architecture of our proposed QRL pipeline for portfolio optimization in the illustrative case of a two-asset, two-time-step environment.

Each horizontal wire corresponds to a qubit. Classical input features (lagged market price information such as $p_{i,t-1}$ and $p_{i,t-2}$) are embedded into the quantum state $|\psi\rangle$ via normalized amplitude embedding, forming the data-encoding layer of the circuit. This encoding maps the classical state of the environment into a coherent quantum representation while remaining hardware efficient and scalable.

Following encoding, the circuit consists of alternating layers of parameterized single-qubit rotations and structured entangling gates. The entanglement pattern is designed to couple both assets and temporal dimensions, enabling the circuit to learn cross-asset dependencies as well as intertemporal relationships. The repeated variational layers constitute the trainable core of both the actor and critic portions of the circuit. In the actor circuit, the final measurement layer extracts Pauli-Z expectation values from a subset of qubits, which are classically normalized to produce a continuous portfolio allocation satisfying the full-investment constraint. In the critic circuit, an analogous readout produces a scalar output in the range $[-1, 1]$, which is interpreted as an estimate of the risk-adjusted portfolio performance, corresponding to the Sharpe ratio associated with the actor's proposed allocation.

The example shown in Figure 3.5 scales directly to larger asset universes and longer observation windows without changing the underlying architecture. Scalability is governed by the amplitude-encoding stage described earlier. Increasing the number of assets or timesteps increases the classical input dimension, which is first expanded by the feature map and then embedded into a quantum state $|\psi\rangle$ using a logarithmic number of qubits. Since the feature map expands the input only by a constant factor and amplitude encoding requires $\mathcal{O}(\log d)$ qubits for a d-dimensional input, the circuit width grows logarithmically as the effective feature space scales linearly, making the experimental design described in the following section feasible.

4 Experimental Design

In the following we describe the experimental design used to assess the effectiveness of the proposed QRL framework in a realistic portfolio optimization setting. Rather than relying on idealized or synthetic benchmarks, we ground our evaluation in historical financial data and formulate portfolio allocation as a continuous control problem that reflects practical investment constraints. States, actions, and rewards are defined to align with standard portfolio management practice, enabling a meaningful comparison between learning-based strategies.

To contextualize the performance of the quantum agents, we benchmark them against static classical baselines and state-of-the-art Deep RL methods operating under identical market conditions. This experimental setup is intended to isolate the impact of the quantum policy representation while ensuring that observed performance differences arise from the learning architecture itself.

4.1 Data and Environment

The dataset consists of 5049 daily closing prices for 15 diverse financial assets, including equities, fixed income instruments, and real assets. Asset returns are computed as logarithmic differences. The full list of assets is provided in Appendix A.3. The data span a period from August 2011 to September 2025, covering multiple market cycles and regimes.

Each environment state comprises a 30-day rolling window of normalized asset prices concatenated with a 7-day forecast obtained using an Auto ARIMA model [Box and Jenkins, 1970], allowing the agent to condition its decisions on predicted short-term market movements in addition to past returns. This hybrid state representation captures both short-term momentum and expected trend behaviour, allowing the agent to condition its policy on recent dynamics as well as forward-looking signals.

All RL approaches considered in this work are inherently dynamic and forward-looking. Portfolio allocations are dynamically rebalanced at a fixed frequency of 30 days. Forward-looking information is incorporated implicitly through the optimization of expected future rewards and explicitly through the forecast window added to the state representation.

Actions are continuous-valued portfolio allocations, where negative values correspond to short positions. At each decision point, the agent outputs a 15-dimensional vector $\mathbf{w}_t$, where $w_{t,i}$ denotes the proportion of capital allocated to asset i at time t. The portfolio is rebalanced every 30 days. Short selling is permitted up to 100 % of the initial portfolio value, reflecting realistic leverage constraints in institutional asset management. Transitions in the environment are governed by observed market returns.

4.2 Objective and Evaluation Metrics

The main performance metric used for evaluation in this study is the Sharpe ratio of the resulting portfolio on the held-out test segment, averaged across the seven folds of our increasing rolling-window time-series cross-validation. All results are computed while taking into account a transaction cost of 0.15 % per trade, corresponding to the transaction costs of a typical large institutional investor [Frazzini et al., 2018].

The Sharpe ratio is computed as the ratio of excess return to volatility, adjusted with a small constant ε in the denominator to prevent division by zero.

$$
\begin{aligned}
\text{Sharpe Ratio} &= \frac{R_p - R_f}{\sigma_p + \varepsilon}, \\
R_p &= \frac{1}{T}\sum_{t=1}^{T} R_{p,t}, \quad R_{p,t} = \sum_{i=1}^{N} p_{t,i}\left(w_{t,i} - \frac{0.15}{100}\,|w_{t,i} - w_{t-1,i}|\right), \\
R_f &= 0.0418 \quad \text{(US 10-Year Treasury yield)}, \\
\sigma_p &= \sqrt{\frac{1}{T-1}\sum_{t=1}^{T}(R_{p,t} - R_p)^2}, \quad \varepsilon = 10^{-7}.
\end{aligned}
$$

A schematic overview of the cross-validation procedure is shown in Figure 4.1. Unlike a fixed-length sliding window, our design follows an expanding-window protocol in which each fold's training set grows monotonically, always incorporating the full historical record available up to the start of that fold. This approach respects causal ordering, reduces estimation noise for later folds, and reflects the operational regime in which an RL policy deployed in markets accumulates information.

Within each fold, the data are partitioned into training, validation, and test subsets. The network parameters are updated exclusively on the training portion, hyperparameter tuning and early stopping rely on the validation portion, and final performance metrics are computed on the strictly unseen test portion.

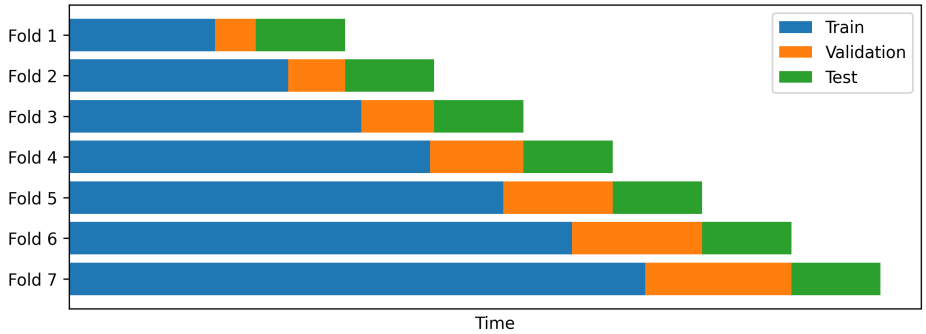

Figure 4.1: The implemented time series cross-validation with 7 folds, each consisting of a training, validation, and test period.

We report the mean and standard deviation of the Sharpe ratio across cross-validation folds to capture not only average performance but also the stability of the learned policies across varying market conditions.

4.3 Baselines and Comparisons

The QRL agent is benchmarked against three baselines. First, the equal-weighted portfolio, which assigns $w_i = \frac{1}{N}$ for all assets and represents a widely used naïve benchmark in portfolio construction for its robustness due to the lack of parameters. Second, MVO, which selects weights that maximize the in-sample Sharpe ratio by brute-force optimization over the admissible allocation space. MVO is theoretically grounded in classical portfolio theory and provides a well-established static standard against which any dynamic strategy should demonstrate value added.

As a comparison, we include classical Deep RL methods: DDPG and a continuous-action version of Deep Q-Learning / DQN as described in Algorithm 4. These serve as representatives of current state-of-the-art RL performance. Importantly, both architectures are evaluated under multiple model sizes to expose the relationship between parameter count, expressivity, runtime, and empirical performance. Including low-capacity (13.5k parameters), medium-capacity (27k parameters), and high-capacity (160k parameters) classical RL alongside QRL models with 30 and 60 parameters allows us to contrast classical and quantum scaling behaviour.

4.4 Training and Deployment Setup

All the classical models are trained on an Nvidia RTX 8000 GPU. For the quantum models, we adopt a hybrid pipeline in which all training is conducted on a noiseless statevector simulator, and the resulting circuits are deployed to a QPU only for inference. After determining the optimal parameters, the optimized parameterized unitary $\mathbf{U}(\boldsymbol{\theta}^\star)$ is executed on a superconducting quantum device to evaluate practical inference behaviour. Specifically, we use the IBM Eagle r3 QPU (127 qubits, 180K CLOPS) to run the converged variational circuit and obtain policy actions via repeated measurement of observable $\hat{B}$ with 10,000 shots. This deployment step enables us to measure real hardware latency, characterize system-level overhead such as qubit initialization and queueing, and assess the extent to which current quantum devices can support online decision-making in a trading context.

This division between training and execution is motivated primarily by the high cost and limited throughput of current cloud-based QPU access. At 12 seconds per forward pass and 96 USD per minute IBM's QPU access makes executing the entire training and hyperparameter optimization prohibitively expensive and slow. A notable downside of this hybrid approach is that statevector simulation scales exponentially with the number of qubits, as opposed to QPU which is practically constant. This restricts the size of the quantum models that can be feasibly trained. Our available GPU imposes a practical upper bound on circuit width of roughly 20 qubits, thereby limiting us to somewhat compact VQCs compared to the 127 qubits that could, in principle, be executed on an IBM Eagle r3. However, training on a statevector simulator provides the benefit that gradients and expectations can be computed exactly, without the sampling noise that would otherwise arise from finite-shot measurement on hardware. As a result, optimization becomes more stable, and convergence is not affected by random variations resulting from the measurement process.

Gradient-based learning is performed using the parameter-shift rule (see Equation 3.2) in combination with the Adam optimizer [Kingma and Ba, 2017]. Hyperparameters are tuned using Bayesian search, with the Sharpe ratio on the validation set defined as the acquisition function. This ensures that the final quantum policies directly target risk-adjusted portfolio performance. Hyperparameters considered for tuning include learning rates, risk and future reward discount factors, and regularization strength. Detailed lists of the hyperparameters and their tuning ranges are provided in Appendices A.1 and A.2.

5 Results

This section examines the results of our empirical evaluation of how quantum agents perform in realistic financial environments and compares their behaviour to established static baselines as well as classical Deep RL methods. Our analysis focuses not only on risk-adjusted performance, but also on robustness across market regimes and efficiency relative to model complexity.

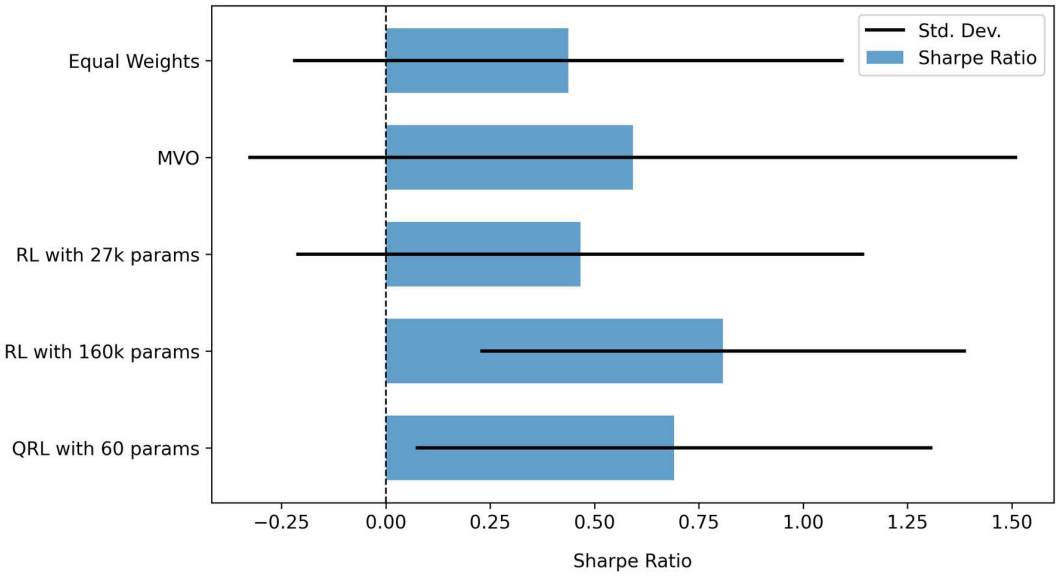

Figure 5.1: Comparison of portfolio Sharpe ratios across models with error bars indicating variability across cross-validation folds.

Figure 5.1 summarizes the Sharpe ratios achieved by all models, including static baselines (equal weights and MVO portfolios), classical Deep RL agents with varying parameter counts, and the proposed QRL agent. Quantum agents consistently outperform classical counterparts when performance is considered relative to parameter count. It is important to emphasize that the error bars reflect variability across cross-validation folds and therefore measure robustness and stability across market regimes, not statistical uncertainty in the Sharpe ratio estimate itself.

Looking at the disaggregated results provided in Table 5.1, the classical models establish a clear scaling trend. The absolute minimum parameter count in our setup is 13,500 parameters, corresponding to neural networks without hidden layers. These minimal-capacity agents perform roughly on par with the equal-weights baseline, confirming that this very limited representational flexibility restricts policy quality. Doubling the model size to 27,000 parameters produces a marginal improvement, whereas the 160k-parameter models reflect the state-of-the-art capacity for classical Deep RL in this environment, achieving the highest Sharpe ratios among all architectures. Nonetheless, their performance gains come at a steep parameter cost, confirming the inefficiency of classical RL when forced to learn complex stochastic policies.

Table 5.1: Disaggregated Portfolio Sharpe Ratios of all Models

	Sharpe Ratio[1]	**Execution Time**
Equal Weights Portfolio	0.4375 (0.6558)	< 1 s[2]
Mean-Variance Optimization	0.5919 (0.9170)	< 1 s[2]
Classical DDPG with 13.5k params	0.4287 (0.6151)	7 s[2]
Classical DQN with 13.5k params	0.4446 (0.6568)	11 s[2]
Classical DDPG with 27k params	0.4384 (0.6905)	10 s[2]
Classical DQN with 27k params	0.4939 (0.6623)	18 s[2]
Classical DDPG with 160k params	0.7926 (0.6276)	14 s[2]
Classical DQN with 160k params	0.8237 (0.5367)	27 s[2]
Quantum DDPG with 30 params	0.4179 (0.6113)	< 1 s (23 min)[3,4]
Quantum DQN with 30 params	0.4776 (0.6650)	< 1 s (43 min)[3,4]
Quantum DDPG with 60 params	0.7281 (0.7395)	< 1 s (23 min)[3,4]
Quantum DQN with 60 params	0.6537 (0.6155)	< 1 s (43 min)[3,4]

[1]Value in parentheses indicates standard deviation of Sharpe ratio across cross-validation folds.
[2]Hardware: Nvidia RTX 8000 GPU. [3]Hardware: IBM Eagle r3 QPU.
[4]Times in parentheses indicate total job duration on cloud-based system.

Quantum models display a different pattern. The absolute minimum quantum parameter count is 30, primarily made possible by amplitude encoding, which compresses high-dimensional portfolio states into a compact quantum wavefunction representation. This compression dramatically reduces qubit requirements while maintaining expressive capacity. While the 30-parameter quantum agents only barely outperform the equal weights baselines, the advantage of quantum representation emerges when the parameter count is increased. Doubling the parameters to 60 brings quantum performance into the same range as classical agents with several orders of magnitude more parameters and consistently outperforms MVO.

Although current hardware constraints limited our experiments to quantum models with at most 60 trainable parameters, these compact architectures already achieve performance comparable to, and in some cases exceeding, that of substantially larger classical models. This result highlights a key advantage of quantum representations in the low-parameter regime: through amplitude encoding and entanglement, VQCs can capture complex dependencies in high-dimensional financial data without requiring large parameter counts. Taken together, these results indicate that compact QRL models can deliver robust risk-adjusted performance in a low-parameter regime, suggesting that it may offer meaningful advantages even before large-scale, fault-tolerant quantum hardware becomes available.

When looking at the disaggregated results, we observe that DQNs generally take longer to execute than DDPG due to the overhead associated with sampling multiple candidate actions to approximate the maximum next action-value in continuous action spaces. Quantum models show moderate variance despite their small parameter footprint, suggesting efficient utilization of representational capacity and a favourable expressivity-to-parameter ratio.

Execution times illustrate the largest operational gap between classical and quantum systems. Classical agents remain extremely fast in inference, with latencies on the order of seconds when executed on GPU hardware. Quantum execution, on the other hand, has two faces: The actual evaluation of a VQC (circuit application, unitary evolution, and measurement) is extremely fast, typically on the order of tens of milliseconds, which is why we report an effective inference time of less than one second for all our quantum models when measured strictly at the circuit-execution level.

The substantial delays we observe in end-to-end cloud inference stem not from the intrinsic speed of superconducting quantum hardware, but from the realities of current quantum cloud computing. On systems such as the IBM Eagle r3, each inference call incurs roughly 12 seconds of latency caused by multi-tenant queueing, repeated qubit initialization, and system-level synchronization steps that must be re-executed at every forward pass because many unrelated users submit heterogeneous circuits to the same device. These overheads accumulate to total job durations of 23 minutes for quantum DDPG and 43 minutes for quantum DQNs, despite the underlying circuits being rapidly executable.

This divergence between intrinsic quantum computation time and practical end-to-end latency therefore reflects less a fundamental limitation of present-day QPU and more a limitation imposed by the economic and infrastructural constraints of current quantum deployments. If a QPU were operated in a dedicated, non-cloud-shared setting—where the exact same circuit structure is executed repeatedly, as would be typical for RL inference—the need for repeated calibration, queuing, and reinitialization would be dramatically reduced. Under such conditions, VQC evaluation would approach its native millisecond timescale.

Despite the latency associated with cloud access, the qualitative advantage of quantum models remains clear. Their high expressivity at extremely small parameter counts, combined with favourable asymptotic scaling that avoids the parameter blow-up characteristic of deep classical models, points toward a compelling future trajectory. Classical inference is much faster under current deployment conditions, but as quantum devices become more affordable and accessible (enabling private or semi-dedicated operation with minimal overhead), or cloud-based latencies are reduced, the efficiency frontier may shift.

6 Conclusion

Prior research has explored quantum methods for portfolio optimization largely in static settings through QUBO-based formulations, while studies in QRL have focused on stylized low-dimensional control problems. Despite its theoretical potential, the application of QRL to complex real-world problems, such as portfolio management, has remained largely unexplored. This work addresses that gap by developing and examining various QRL approaches for dynamic portfolio optimization. By introducing and benchmarking a fully quantum RL framework for sequential decision-making based on VQCs, we provide a comprehensive assessment of how quantum agents behave in a meaningful financial setting.

Our QRL agents effectively learn adaptive trading policies that respond to market dynamics. The presented results show that quantum DDPG and quantum DQNs achieve risk-adjusted performance comparable to, and in some cases exceeding, that of their classical counterparts with several orders of magnitude more parameters. This efficiency arises from the structural advantages of quantum representation, which enables compact yet expressive policy and value function approximators.

At the same time, our experiments highlight the realities of quantum computing today. While circuit execution on superconducting hardware is extremely fast, cloud-based quantum access introduces substantial latency because many users submit heterogeneous jobs to the same device. These factors inflate end-to-end inference times from miliseconds to minutes even for shallow circuits. Importantly, this is not a hardware limitation in the strict sense: if QPUs were operated in dedicated environments, where repeated execution of similar circuits eliminates redundant preparation overhead, inference would approach its native millisecond timescale.

Taken together, these findings place us at an instructive moment for QRL and Quantum Machine Learning in general. On the algorithmic side, our results demonstrate that QRL already delivers meaningful performance and that quantum expressivity can be leveraged in sequential financial decision-making. On the hardware side, however, practical deployment is constrained by access models and economic factors. As quantum devices become more affordable, more stable, and more readily available for dedicated use, the gap between theoretical and practical quantum capability is likely to shrink. QRL therefore has the potential to transition from a theoretically appealing alternative to a practically viable solution for complex dynamic decision problems in finance and beyond.

Acknowledgements

Stefan Lessmann acknowledges financial support through the project “AI4EFin AI for Energy Finance”, contract number CF162/15.11.2022, financed under Romania’s National Recovery and Resilience Plan, Apel nr. PNRR-III-C9-2022-I8.

References

Harry Markowitz. Portfolio Selection. *The Journal of Finance*, 7(1):77–91, 1952. ISSN 1540-6261. doi:10.1111/j.1540-6261.1952.tb01525.x. URL https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1952.tb01525.x. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1540-6261.1952.tb01525.x.

Fischer Black and Robert Litterman. Global Portfolio Optimization. *Financial Analysts Journal*, 48(5):28–43, 1992. ISSN 0015-198X. URL https://www.jstor.org/stable/4479577.

Frank J. Fabozzi, Peter N. Kolm, Dessislava A. Pachamanova, and Sergio M. Focardi. *Robust Portfolio: Optimization and Management*. John Wiley & Sons, Inc., January 2012. ISBN 978-0-471-92122-6. URL doi.org/10.1002/9781119202172.

Giancarlo Martínez Salirrosas, Jinglun Gao, Arthur Yu, and Anish Ravi Verma. Market informed portfolio optimization methods with hybrid quantum computing. *Review of Financial Economics*, 43(1):62–77, 2025. ISSN 1873-5924. doi:10.1002/rfe.1219. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/rfe.1219. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/rfe.1219.

Ahti Salo, Michalis Doumpos, Juuso Liesiö, and Constantin Zopounidis. Fifty years of portfolio optimization. *European Journal of Operational Research*, 318(1):1–18, October 2024. ISSN 0377-2217. doi:10.1016/j.ejor.2023.12.031. URL https://www.sciencedirect.com/science/article/pii/S0377221723009827.

Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep Reinforcement Learning for Trading. *The Journal of Financial Data Science*, 2(2):25–40, April 2020. ISSN 2640-3943. doi:10.3905/jfds.2020.1.030. URL http://pm-research.com/lookup/doi/10.3905/jfds.2020.1.030.

Román Orús, Samuel Mugel, and Enrique Lizaso. Quantum computing for finance: Overview and prospects. *Reviews in Physics*, 4:100028, November 2019. ISSN 2405-4283. doi:10.1016/j.revip.2019.100028. URL https://www.sciencedirect.com/science/article/pii/S2405428318300571.

Daniel J. Egger, Claudio Gambella, Jakub Marecek, Scott McFaddin, Martin Mevissen, Rudy Raymond, Andrea Simonetto, Stefan Woerner, and Elena Yndurain. Quantum Computing for Finance: State of the Art and Future Prospects. *IEEE Transactions on Quantum Engineering*, 1:1–24, 2020. ISSN 2689-1808. doi:10.1109/TQE.2020.3030314. URL http://arxiv.org/abs/2006.14510. arXiv:2006.14510 [quant-ph].

Giuseppe Buonaiuto, Francesco Gargiulo, Giuseppe De Pietro, Massimo Esposito, and Marco Pota. Best practices for portfolio optimization by quantum computing, experimented on real quantum devices. *Scientific Reports*, 13(1):19434, November 2023. ISSN 2045-2322. doi:10.1038/s41598-023-45392-w. URL https://www.nature.com/articles/s41598-023-45392-w.

Nico Meyer, Christian Ufrecht, Maniraman Periyasamy, Daniel D. Scherer, Axel Plinge, and Christopher Mutschler. A Survey on Quantum Reinforcement Learning, March 2024. URL http://arxiv.org/abs/2211.03464. arXiv:2211.03464 [quant-ph].

Lenny Putri Yulianti and Kridanto Surendro. Implementation of Quantum Annealing: A Systematic Review. *IEEE Access*, 10:73156–73177, 2022. ISSN 2169-3536. doi:10.1109/ACCESS.2022.3188117. URL https://ieeexplore.ieee.org/document/9813715/.

Vikram Sehrawat. Quantum Computing: Algorithms and Applications in Optimization Problems. *Journal of Quantum Science and Technology*, 1(2):18–22, July 2024. ISSN 3048-6351. doi:10.36676/jqst.v1.i2.11. URL https://jqst.mindsynk.org/index.php/j/article/view/11.

Deborah Volpe, Giacomo Orlandi, and Giovanna Turvani. Improving the Solving of Optimization Problems: A Comprehensive Review of Quantum Approaches. *Quantum Reports*, 7(1):3, March 2025. ISSN 2624-960X. doi:10.3390/quantum7010003. URL https://www.mdpi.com/2624-960X/7/1/3.

Gary Kochenberger, Jin-Kao Hao, Fred Glover, Zhipeng Lü, Haibo Wang, and Yang Wang. The unconstrained binary quadratic programming problem: A survey. *Journal of Combinatorial Optimization*, 28, July 2014. doi:10.1007/s10878-014-9734-0.

Fred Glover, Gary Kochenberger, and Yu Du. A Tutorial on Formulating and Using QUBO Models, November 2019. URL http://arxiv.org/abs/1811.11538. arXiv:1811.11538 [cs].

Frank Phillipson and Harshil Singh Bhatia. Portfolio Optimisation Using the D-Wave Quantum Annealer. In Maciej Paszynski, Dieter Kranzlmüller, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M. A. Sloot, editors, *Computational Science – ICCS 2021*, pages 45–59, Cham, 2021. Springer International Publishing. ISBN 978-3-030-77980-1. doi:10.1007/978-3-030-77980-1_4.

Sebastian Brandhofer, Daniel Braun, Vanessa Dehn, Gerhard Hellstern, Matthias Hüls, Yanjun Ji, Ilia Polian, Amandeep Singh Bhatia, and Thomas Wellens. Benchmarking the performance of portfolio optimization with QAOA. *Quantum Information Processing*, 22(1):25, December 2022. ISSN 1573-1332. doi:10.1007/s11128-022-03766-5. URL https://doi.org/10.1007/s11128-022-03766-5.

Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A Quantum Approximate Optimization Algorithm, November 2014. URL http://arxiv.org/abs/1411.4028. arXiv:1411.4028 [quant-ph].

Ramin Fakhimi and Hamidreza Validi. Quantum Approximate Optimization Algorithm (QAOA). In *Encyclopedia of Optimization*, pages 1–7. Springer, Cham, 2023. ISBN 978-3-030-54621-2. doi:10.1007/978-3-030-54621-2_854-1. URL https://link.springer.com/rwe/10.1007/978-3-030-54621-2_854-1.

Eric Stopfer and Friedrich Wagner. Quantum Portfolio Optimization: An Extensive Benchmark, September 2025. URL http://arxiv.org/abs/2509.17876. arXiv:2509.17876 [quant-ph].

Dylan Herman, Ruslan Shaydulin, Yue Sun, Shouvanik Chakrabarti, Shaohan Hu, Pierre Minssen, Arthur Rattew, Romina Yalovetzky, and Marco Pistoia. Constrained Optimization via Quantum Zeno Dynamics. *Communications Physics*, 6(1):219, August 2023. ISSN 2399-3650. doi:10.1038/s42005-023-01331-9. URL http://arxiv.org/abs/2209.15024. arXiv:2209.15024 [quant-ph].

Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5(1):4213, July 2014. ISSN 2041-1723. doi:10.1038/ncomms5213. URL https://www.nature.com/articles/ncomms5213.

Anbang Wang, Zhonggang Lv, Zhenyuan Ma, Dunbo Cai, and Zhihong Zhang. Achieving High-Quality Portfolio Optimization with the Variational Quantum Eigensolver, August 2025a. URL http://arxiv.org/abs/2508.18625. arXiv:2508.18625 [quant-ph].

Ioannis Kolotouros and Petros Wallden. An evolving objective function for improved variational quantum optimisation. *Physical Review Research*, 4(2):023225, June 2022. ISSN 2643-1564. doi:10.1103/PhysRevResearch.4.023225. URL http://arxiv.org/abs/2105.11766. arXiv:2105.11766 [quant-ph].

Chi-Chun Chen, San-Lin Chung, and Hsi-Sheng Goan. Black-Litterman Portfolio Optimization with Noisy Intermediate-Scale Quantum Computers, December 2023. URL http://arxiv.org/abs/2312.00892. arXiv:2312.00892 [quant-ph].

Kamila Zaman, Alberto Marchisio, Muhammad Kashif, and Muhammad Shafique. PO-QA: A Framework for Portfolio Optimization using Quantum Algorithms. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 1397–1403, Montreal, QC, Canada, September 2024. IEEE. ISBN 979-8-3315-4137-8. doi:10.1109/QCE60285.2024.00166. URL https://ieeexplore.ieee.org/document/10821300/.

Tadashi Kadowaki and Hidetoshi Nishimori. Quantum annealing in the transverse Ising model. *Physical Review E*, 58(5):5355–5363, November 1998. doi:10.1103/PhysRevE.58.5355. URL https://link.aps.org/doi/10.1103/PhysRevE.58.5355.

Arnab Das and Bikas K. Chakrabarti. Colloquium: Quantum annealing and analog quantum computation. *Reviews of Modern Physics*, 80(3):1061–1081, September 2008. doi:10.1103/RevModPhys.80.1061. URL https://link.aps.org/doi/10.1103/RevModPhys.80.1061.

Zhijie Tang, Alex Lu Dou, and Arit Kumar Bishwas. Comparative analysis of diverse methodologies for portfolio optimization leveraging quantum annealing techniques, July 2024. URL http://arxiv.org/abs/2403.02599. arXiv:2403.02599 [quant-ph].

Gili Rosenberg, Poya Haghnegahdar, Phil Goddard, Peter Carr, Kesheng Wu, and Marcos López de Prado. Solving the Optimal Trading Trajectory Problem Using a Quantum Annealer. *IEEE Journal of Selected Topics in Signal Processing*, 10(6):1053–1060, September 2016. ISSN 1941-0484. doi:10.1109/JSTSP.2016.2574703. URL https://ieeexplore.ieee.org/document/7482755.

Davide Venturelli and Alexei Kondratyev. Reverse quantum annealing approach to portfolio optimization problems. *Quantum Machine Intelligence*, 1(1):17–30, May 2019. ISSN 2524-4914. doi:10.1007/s42484-019-00001-w. URL https://doi.org/10.1007/s42484-019-00001-w.

Jeffrey Cohen, Alex Khan, and Clark Alexander. Portfolio Optimization of 40 Stocks Using the DWave Quantum Annealer, July 2020. URL http://arxiv.org/abs/2007.01430. arXiv:2007.01430 [q-fin].

Samuel Mugel, Carlos Kuchkovsky, Escolástico Sánchez, Samuel Fernández-Lorenzo, Jorge Luis-Hita, Enrique Lizaso, and Román Orús. Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks. *Physical Review Research*, 4(1):013006, January 2022. ISSN 2643-1564. doi:10.1103/PhysRevResearch.4.013006. URL https://link.aps.org/doi/10.1103/PhysRevResearch.4.013006.

Ginés Carrascal de las Heras, Paula Hernamperez Manso, Guillermo Botella, and Alberto Antonio del Barrio. Backtesting quantum computing algorithms for portfolio optimization. April 2023. URL https://www.authorea.com/doi/full/10.36227/techrxiv.22679857.v1?commit=c03a9fa03d8db724f15ba7db676e32347c477674.

Naman Jain, Ankit Khandelwal, and M Girish Chandra. Efficient and Flexible Annealer-Gate Hybrid Model for Solving Large-Scale Portfolio Optimization. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 02, pages 286–287, September 2023. doi:10.1109/QCE57702.2023.10245. URL https://ieeexplore.ieee.org/document/10313924/.

Esteban Aguilera, Jins de Jong, Frank Phillipson, Skander Taamallah, and Mischa Vos. Multi-Objective Portfolio Optimization Using a Quantum Annealer. *Mathematics*, 12(9):1291, January 2024. ISSN 2227-7390. doi:10.3390/math12091291. URL https://www.mdpi.com/2227-7390/12/9/1291.

Francesco Catalano, Laura Nasello, and Daniel Guterding. Quantum computing approach to realistic ESG-friendly stock portfolios, April 2024. URL https://arxiv.org/abs/2404.02582v1.

Vu Truc Quynh, Vu Tuan Hai, Le Vu Trung Duong, Pham Hoai Luan, and Yasuhiko Nakashima. A Quantum Circuit Design for Quantum Portfolio Optimization Problem. In *2024 International Technical Conference on Circuits/Systems, Computers, and Communications (ITC-CSCC)*, pages 1–6, July 2024. doi:10.1109/ITC-CSCC62988.2024.10628188. URL https://ieeexplore.ieee.org/document/10628188/.

Yao-Hsin Chou, Ching-Hsuan Wu, Pei-Shin Huang, Jyun-Yi Shen, Shu-Yu Kuo, Sy-Yen Kuo, and Ching-Ray Chang. Exploring Quantum Annealing for Enhanced International Financial Stock Portfolio Management. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 01, pages 250–258, September 2024. doi:10.1109/QCE60285.2024.00038. URL https://ieeexplore.ieee.org/document/10821420/.

Shengbin Wang, Peng Wang, Guihui Li, Shubin Zhao, Dongyi Zhao, Jing Wang, Yuan Fang, Menghan Dou, Yongjian Gu, Yu-Chun Wu, and Guo-Ping Guo. Variational quantum eigensolver with linear depth problem-inspired ansatz for solving portfolio optimization in finance. *Science China Information Sciences*, 68(8):180504, July 2025b. ISSN 1869-1919. doi:10.1007/s11432-024-4185-1. URL https://doi.org/10.1007/s11432-024-4185-1.

Wolfgang Sakuler, Johannes M. Oberreuter, Riccardo Aiolfi, Luca Asproni, Branislav Roman, and Jürgen Schiefer. A real-world test of portfolio optimization with quantum annealing. *Quantum Machine Intelligence*, 7(1):43, March 2025. ISSN 2524-4914. doi:10.1007/s42484-025-00268-2. URL https://doi.org/10.1007/s42484-025-00268-2.

Samuel Yen-Chi Chen, Chao-Han Huck Yang, Jun Qi, Pin-Yu Chen, Xiaoli Ma, and Hsi-Sheng Goan. Variational Quantum Circuits for Deep Reinforcement Learning, July 2020. URL http://arxiv.org/abs/1907.00397. arXiv:1907.00397 [cs].

Owen Lockwood and Mei Si. Reinforcement Learning with Quantum Variational Circuit. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 16(1):245–251, October 2020. ISSN 2334-0924. doi:10.1609/aiide.v16i1.7437. URL https://ojs.aaai.org/index.php/AIIDE/article/view/7437.

Sofiene Jerbi, Casper Gyurik, Simon C. Marshall, Hans J. Briegel, and Vedran Dunjko. Parametrized quantum policies for reinforcement learning, December 2021. URL http://arxiv.org/abs/2103.05577. arXiv:2103.05577 [quant-ph].

Hsin-Yi Lin, Samuel Yen-Chi Chen, Huan-Hsin Tseng, and Shinjae Yoo. Quantum Reinforcement Learning by Adaptive Non-local Observables, July 2025. URL http://arxiv.org/abs/2507.19629. arXiv:2507.19629 [quant-ph].

El Amine Cherrat, Snehal Raj, Iordanis Kerenidis, Abhishek Shekhar, Ben Wood, Jon Dee, Shouvanik Chakrabarti, Richard Chen, Dylan Herman, Shaohan Hu, Pierre Minssen, Ruslan Shaydulin, Yue Sun, Romina Yalovetzky, and Marco Pistoia. Quantum Deep Hedging. *Quantum*, 7:1191, November 2023. ISSN 2521-327X. doi:10.22331/q-2023-11-29-1191. URL http://arxiv.org/abs/2303.16585. arXiv:2303.16585 [quant-ph].

Junzheng Yang. Apply Deep Reinforcement Learning with Quantum Computing on the Pricing of American Options. *World Scientific Book Chapters*, pages 675–694, 2023. URL https://ideas.repec.org//h/wsi/wschap/9789811267505_0050.html.

Michael A. Nielsen and Isaac L. Chuang. Quantum Computation and Quantum Information: 10th Anniversary Edition, December 2010. URL https://www.cambridge.org/highereducation/books/quantum-computation-and-quantum-information/01E10196D0A682A6AEFFEA52D53BE9AE. ISBN: 9780511976667.

Ben Freiman. The hydrogen molecule and the Hartree-Fock method, January 2024. URL https://fabianfaulstich.github.io/QMSeminar_2024/documents/QuantumManyBody_BenFreiman.pdf.

Ryszard Horodecki, Paweł Horodecki, Michał Horodecki, and Karol Horodecki. Quantum entanglement. *Reviews of Modern Physics*, 81(2):865–942, June 2009. ISSN 0034-6861, 1539-0756. doi:10.1103/RevModPhys.81.865. URL https://link.aps.org/doi/10.1103/RevModPhys.81.865.

A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. Smolin, and H. Weinfurter. Elementary gates for quantum computation. *Physical Review A*, 52(5):3457–3467, November 1995. ISSN 1050-2947, 1094-1622. doi:10.1103/PhysRevA.52.3457. URL http://arxiv.org/abs/quant-ph/9503016. arXiv:quant-ph/9503016.

Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997. ISSN 0097-5397, 1095-7111. doi:10.1137/S0097539795293172. URL http://arxiv.org/abs/quant-ph/9508027. arXiv:quant-ph/9508027.

Lov K. Grover. Quantum Mechanics helps in searching for a needle in a haystack. *Physical Review Letters*, 79(2):325–328, July 1997. ISSN 0031-9007, 1079-7114. doi:10.1103/PhysRevLett.79.325. URL http://arxiv.org/abs/quant-ph/9706033. arXiv:quant-ph/9706033.

Alán Aspuru-Guzik, Anthony D. Dutoi, Peter J. Love, and Martin Head-Gordon. Simulated Quantum Computation of Molecular Energies. *Science*, 309(5741):1704–1707, September 2005. ISSN 0036-8075, 1095-9203. doi:10.1126/science.1113479. URL http://arxiv.org/abs/quant-ph/0604193. arXiv:quant-ph/0604193.

John Preskill. Quantum Computing in the NISQ era and beyond. *Quantum*, 2:79, August 2018. doi:10.22331/q-2018-08-06-79. URL https://quantum-journal.org/papers/q-2018-08-06-79/.

Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, November 2019. ISSN 2058-9565. doi:10.1088/2058-9565/ab4eb5. URL https://dx.doi.org/10.1088/2058-9565/ab4eb5.

Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99(3):032331, March 2019. ISSN 2469-9926, 2469-9934. doi:10.1103/PhysRevA.99.032331. URL https://link.aps.org/doi/10.1103/PhysRevA.99.032331.

M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, September 2021. ISSN 2522-5820. doi:10.1038/s42254-021-00348-9. URL https://www.nature.com/articles/s42254-021-00348-9.

Maria Schuld, Ilya Sinayskiy, and Francesco Petruccione. The quest for a Quantum Neural Network. *Quantum Information Processing*, 13(11):2567–2586, November 2014. ISSN 1573-1332. doi:10.1007/s11128-014-0809-8. URL https://doi.org/10.1007/s11128-014-0809-8.

Maria Schuld, Alex Bocharov, Krysta Svore, and Nathan Wiebe. Circuit-centric quantum classifiers. *Physical Review A*, 101(3):032308, March 2020. ISSN 2469-9926, 2469-9934. doi:10.1103/PhysRevA.101.032308. URL http://arxiv.org/abs/1804.00633. arXiv:1804.00633 [quant-ph].

Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. The effect of data encoding on the expressive power of variational quantum machine learning models. *Physical Review A*, 103(3):032430, March 2021. ISSN 2469-9926, 2469-9934. doi:10.1103/PhysRevA.103.032430. URL http://arxiv.org/abs/2008.08605. arXiv:2008.08605 [quant-ph].

Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, March 2019. ISSN 1476-4687. doi:10.1038/s41586-019-0980-2. URL https://www.nature.com/articles/s41586-019-0980-2.

Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum Algorithm for Linear Systems of Equations. *Physical Review Letters*, 103(15):150502, October 2009. doi:10.1103/PhysRevLett.103.150502. URL https://link.aps.org/doi/10.1103/PhysRevLett.103.150502.

Mikko Möttönen, Juha J. Vartiainen, Ville Bergholm, and Martti M. Salomaa. Transformation of quantum states using uniformly controlled rotations, July 2004. URL http://arxiv.org/abs/quant-ph/0407010. arXiv:quant-ph/0407010.

Martin Plesch and Časlav Brukner. Quantum-state preparation with universal gate decompositions. *Physical Review A*, 83(3):032302, March 2011. ISSN 1050-2947, 1094-1622. doi:10.1103/PhysRevA.83.032302. URL http://arxiv.org/abs/1003.5760. arXiv:1003.5760 [quant-ph].

K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii. Quantum circuit learning. *Physical Review A*, 98(3): 032309, September 2018. doi:10.1103/PhysRevA.98.032309. URL https://link.aps.org/doi/10.1103/PhysRevA.98.032309.

Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9(1):4812, November 2018. ISSN 2041-1723. doi:10.1038/s41467-018-07090-4. URL https://www.nature.com/articles/s41467-018-07090-4.

Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957. ISBN 978-0-691-07951-6.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2015.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3):279–292, May 1992. ISSN 1573-0565. doi:10.1007/BF00992698. URL https://doi.org/10.1007/BF00992698.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015. ISSN 1476-4687. doi:10.1038/nature14236. URL https://www.nature.com/articles/nature14236.

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning, July 2019. URL http://arxiv.org/abs/1509.02971. arXiv:1509.02971 [cs].

Vijay Konda and John Tsitsiklis. Actor-Critic Algorithms. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://papers.nips.cc/paper_files/paper/1999/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization, January 2017. URL http://arxiv.org/abs/1412.6980. arXiv:1412.6980 [cs].

George Box and Gwilym Jenkins. *Time Series Analysis: Forecasting and Control*. Palgrave Macmillan UK, London, 1970. ISBN 978-1-137-29126-4. doi:10.1057/9781137291264_6. URL https://doi.org/10.1057/9781137291264_6.

Andrea Frazzini, Ronen Israel, and Tobias J. Moskowitz. Trading Costs, April 2018. URL https://papers.ssrn.com/abstract=3229719.

Appendix

A.1 Fixed Hyperparameters

- Lookback window: 30 days
- Forecast window: 7 days
- Soft-update with coefficient $\tau = 0.005$
- Epochs: 50, with early stopping enabled at patience 10
- Number of DQN next action samples: 10
- Replay buffer size: unlimited
- Number of shots per QPU circuit evaluation: 10,000

A.2 Hyperparameter Tuning Ranges

- Actor and critic learning rates $\in [10^{-4}, 10^{-1}]$, log scale
- λ (L2 regularization parameter) $\in [10^{-6}, 10^{-1}]$, log scale
- Risk preference $\in [-1, -10^{-2}]$, linear scale
- γ (discount factor for future rewards) $\in [10^{-3}, 10^{-1}]$, log scale
- Optimizer $\in \{\text{Adam}, \text{SGD}\}$

A.3 Full List of Assets

- AAPL (Apple Inc.)
- EFA (iShares MSCI EAFE ETF)
- GLD (SPDR Gold Shares)
- IWM (iShares Russell 2000 ETF)
- JNJ (Johnson & Johnson)
- JPM (JPMorgan Chase & Co.)
- LQD (iShares iBoxx $ Investment Grade Corporate Bond ETF)
- MSFT (Microsoft Corporation)
- QQQ (Invesco QQQ Trust)
- SPY (SPDR S&P 500 ETF)
- TLT (iShares 20+ Year Treasury Bond ETF)
- USO (United States Oil Fund, LP)
- XLF (State Street Financial Select Sector SPDR ETF)
- XLV (State Street Health Care Select Sector SPDR ETF)
- XOM (Exxon Mobil Corporation)